

Prompt2Task: Automating UI Tasks on Smartphones from Textual Prompts

TIAN HUANG, Department of Computer Science and Technology, Tsinghua University, China
CHUN YU*, Department of Computer Science and Technology, Tsinghua University, China
WEINAN SHI, Department of Computer Science and Technology, Tsinghua University, China
ZIJIAN PENG, Department of Computer Science and Technology, Tsinghua University, China
DAVID YANG, Department of Computer Science and Technology, Tsinghua University, China
WEIQI SUN, Department of Public Administration, Sichuan University, China
YUANCHUN SHI, Department of Computer Science and Technology, Tsinghua University, Qinghai University, China

UI task automation enables efficient task execution by simulating human interactions with graphical user interfaces (GUIs), without modifying the existing application code. However, its broader adoption is constrained by the need for expertise in both scripting languages and workflow design. To address this challenge, we present Prompt2Task, a system designed to comprehend various task-related textual prompts (e.g., goals, procedures), thereby generating and performing the corresponding automation tasks. Prompt2Task incorporates a suite of intelligent agents that mimic human cognitive functions, specializing in interpreting user intent, managing external information for task generation, and executing operations on smartphones. The agents can learn from user feedback and continuously improve their performance based on the accumulated knowledge. Experimental results indicated a performance jump from a 22.28% success rate in the baseline to 95.24% with Prompt2Task, requiring an average of 0.69 user interventions for each new task. Prompt2Task presents promising applications in fields such as tutorial creation, smart assistance, and customer service.

CCS Concepts: • **Human-centered computing** → *Ubiquitous and mobile computing systems and tools*; Graphical user interfaces; **Interactive systems and tools**.

Additional Key Words and Phrases: UI task automation, UI navigation, natural language, UI understanding, large language models

ACM Reference Format:

Tian Huang, Chun Yu, Weinan Shi, Zijian Peng, David Yang, Weiqi Sun, and Yuanchun Shi. 2024. Prompt2Task: Automating UI Tasks on Smartphones from Textual Prompts. 1, 1 (March 2024), 46 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*Corresponding author.

Authors' addresses: Tian Huang, ht20@mails.tsinghua.edu.cn, Department of Computer Science and Technology, Tsinghua University, Beijing, China; Chun Yu, Department of Computer Science and Technology, Tsinghua University, Beijing, China, chunyu@tsinghua.edu.cn; Weinan Shi, Department of Computer Science and Technology, Tsinghua University, Beijing, China, swn@mail.tsinghua.edu.cn; Zijian Peng, Department of Computer Science and Technology, Tsinghua University, Beijing, China, pzj21@mails.tsinghua.edu.cn; David Yang, Department of Computer Science and Technology, Tsinghua University, Beijing, China, ydw22@mails.tsinghua.edu.cn; Weiqi Sun, Department of Public Administration, Sichuan University, Sichuan, China, weiqiviki999@gmail.com; Yuanchun Shi, Department of Computer Science and Technology, Tsinghua University, Qinghai University, Beijing, China, shiyc@tsinghua.edu.cn.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2024 Copyright held by the owner/author(s).
XXXX-XXXX/2024/3-ART
<https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

UI task automation has emerged as a pivotal technology, significantly boosting productivity and operational efficiency [44]. Its ability to simulate human interactions on the graphical user interface (GUI) allows for broad compatibility with third-party applications, without necessitating modifications to existing code [17, 38]. However, task automation primarily relies on predefined operation sequences [29, 63]. Customizing these sequences often requires specialized skills in scripting languages and workflow design, which is a significant barrier to adoption, particularly for users without a technical background [1]. Additionally, the fixed operation sequences have difficulty adapting to the frequently updated GUI, making them susceptible to becoming quickly outdated.

To address this gap, we introduce Prompt2Task, which employs a text-prompt-driven task automation approach implemented on smartphones. Prompt2Task is capable of accepting unrestricted textual descriptions for UI tasks—from concrete operational instructions like “open settings, locate WLAN, and...” to high-level functional requests such as “change the ringtone”. Prompt2Task then automatically translates these textual prompts into operation sequences on GUI, freeing users from the complexity of designing automation scripts themselves.

Prompt2Task serves as an intelligent navigation system on smartphones. It interprets user intent from textual descriptions and matches them to its internal “map” for route planning and decision-making, thereby automatically generating task flows. Throughout the process, Prompt2Task faces several challenges. Firstly, it needs to accommodate diverse and unclear textual inputs, requiring Prompt2Task to infer intentions from vague or incomplete instructions [82]. Secondly, the “map” may be incomplete, especially lacking detailed route information for less common tasks or newly added application functions [34]. This issue is difficult to address even with state-of-the-art Large Language Models (LLMs), which may generate incorrect or misleading information in the GUI domain [36, 76]. Thirdly, it is challenging to adapt to frequent updates in interfaces and applications; as they evolve, the routes Prompt2Task relies on may also require updates [73]. These challenges

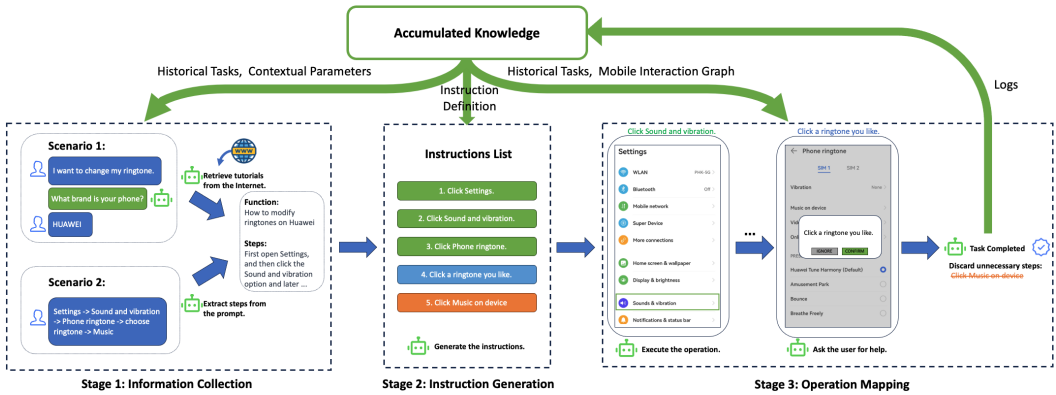


Fig. 1. The workflow of Prompt2Task. In the first stage, users input task-related textual prompts, and the agent collects relevant textual information to plan the task. In the second stage, the agent converts the text into formally defined instructions. In the third stage, the agent maps these instructions to operations, ultimately executing the automation task on the smartphone. Users can intervene in the agents’ decisions, especially when help is requested. Blue arrows guide the process from prompts to task automation, while green arrows indicate the knowledge accumulation through user interactions, thereby enhancing the future performance of the agents.

span across different stages of the process, requiring the application of various types of intelligence and knowledge to address them effectively.

To meet these challenges, we employ a multi-agent framework [58] for Prompt2Task. The framework allocates specialized agents to sub-tasks within different stages of the process, with each agent emulating specific human cognitive functions. As illustrated in Figure 1, Prompt2Task spans three stages—from **information collection** and **instruction generation** to **operation mapping**. Specifically, **information collection** involves gathering all necessary descriptions of functions and steps for the automation task. **Instruction generation** entails creating formalized instructions to guide operations, and **operation mapping** refers to translating text-based instructions into operations on smartphones to ultimately complete the task.

Two core elements are critical for Prompt2Task’s performance: a comprehensive knowledge base [73], which is our smartphone “map”, and stable decision-making [93], which ensures the agents’ efficient and collaborative task execution. Therefore, we create an evolving knowledge base that expands as it accumulates data and user interactions. We introduce three key agents to meet the specific knowledge demands of UI task automation: the *Analysis Agent*, which identifies user intents and actively fills in missing contextual parameters from user inputs; the *Retrieval Agent*, which sources online tutorials to guide task automation; and the *Grounding Agent*, which conducts in-depth semantic analysis of mobile GUI for better alignment with textual inputs and supports unplanned operations upon updates or errors. To enhance decision-making, the *Assessment Agent* scrutinizes decisions made by other agents, serving as an internal audit mechanism to improve accuracy and reliability. Beyond internal audit, the agents are seamlessly integrated into a user interface, allowing for user oversee and intervention to enhance decisions. User intervention is optional and involves chatting, selecting, editing, or demonstrating to modify and confirm the agents’ decisions. User usage and intervention create a feedback loop for continuous knowledge accumulation and system improvement, thereby optimizing efficiency for future tasks.

To evaluate Prompt2Task, we conducted a performance evaluation with 2,500 textual prompts crowd-sourced. The results showed that Prompt2Task increased the task success rate from a baseline of 22.28% to 95.24%, requiring only an average of 0.69 user interventions for each new task. Moreover, it was proved that the accumulation of user usage and intervention improved system performance. Furthermore, a user study involving both skilled and unskilled smartphone users confirmed Prompt2Task’s effectiveness in assistance and its low interaction cost. Due to the low-cost translation from text to automation tasks, Prompt2Task has promising applications in generating interactive tutorials, enhancing smart assistance, and improving online customer service.

Specifically, the contributions of this work are three-fold.

- We propose Prompt2Task, a multi-agent system that accepts unrestricted textual expressions of task requirements and automates corresponding UI tasks. To improve the success rate, Prompt2Task incorporates several innovative strategies: retrieving online tutorials, integrating necessary user interventions, and continuously improving agent capabilities through the accumulation of knowledge from use.
- We propose an effective approach to minimize the cost of user interventions. Prompt2Task presents results stage-by-stage, allowing users to intuitively interact with the system through dialogues, selections, edits, and demonstrations. A dedicated agent employs GPT-4 to evaluate the confidence of predicted operations, requesting user assistance only when necessary.

- We create a Chinese dataset¹ that covers a wide range of UI tasks and collects diverse user descriptions of the tasks. The dataset includes 2,500 textual prompts, spanning 100 tasks across 10 application domains, with a total of 533 instructions and 543 operations.

2 RELATED WORK

Our work intersects with four research areas: prior UI task automation systems, the knowledge required for UI task automation, LLM-driven multi-agent frameworks, and incorporating user feedback for agent improvement.

2.1 Prior UI Task Automation Systems

Prior UI task automation systems on smartphones typically work by replaying predefined interaction flows on graphical user interfaces, thereby completing the automation tasks. Early systems like SUGILITE [44] and APPINITE [45] utilized programming by demonstration to enable users to create interaction flows. However, for complex or unfamiliar functions, creating flows is challenging, especially for beginners and the elderly, who may lack essential task execution skills [14, 37]. Subsequent developments introduce interactive, context-aware tutorials for GUI applications [26, 42, 48, 62, 93]. However, these systems primarily rely on pre-configured flows by developers [78]. When accommodating new tasks or when interface updates cause the original flows outdated [85], these systems require users to have specialized skills in script design [83].

Recent efforts have focused on translating natural language into operation sequences on smartphones. Notable contributions include Branavan et al. [9] and Li et al. [49]. However, they require detailed instructions for each step, necessitating that users have a thorough understanding of task execution. While there are systems that support general task requests, limitations exist, such as UGIF's reliance on a pre-collected tutorial dataset [73] and Voicify's dependence on app developers' implementation of deep links [75], failing to meet a wide range of user needs. Additionally, they often function as "black boxes", which can autonomously complete tasks when functioning correctly but may also produce uncontrollable and erroneous results [73].

2.2 Knowledge Required for UI Task Automation

Successful automation of smartphone tasks demands a comprehensive understanding of diverse knowledge domains. For the input, tutorial datasets need to be sufficiently rich to support the diversity of user needs. Regarding the output, a nuanced grasp of mobile semantics is required to ensure robust execution on smartphones.

Addressing user input presents challenges in accuracy and scope. User inputs are often vague [35], requiring supplementary contextual information for accurate recognition of user intent [27]. In terms of scope, existing UI trace datasets such as Rico [17], UGIF-DataSet [73], and PixelHelp [49] are pre-collected and they are insufficient to meet a wide array of dynamic user needs [6]. AutoVCI [57] and Kite [47] can only recognize user intents limited to predefined UI trace collections. When using search engines or AI tools like GPT for tutorial information, the results can be irrelevant, outdated [73], fabricated [66], or low-quality [10].

The other focus is the understanding of mobile GUI semantics, which is crucial for widget identification and context comprehension. A wealth of research focuses on providing semantic text annotations for UI elements [13, 28, 50, 76]. Other research efforts include generating full-page text descriptions [77], image-to-HTML conversion [39], and GUI embedding [5, 46]. Translating GUI images and hierarchical information into natural language descriptions while preserving key

¹The dataset is at <https://github.com/PromptRPA/Prompt2TaskDataset>.

task-relevant information is crucial for **operation mapping**, as it enables textual instructions to align effectively with GUI operations.

2.3 LLM-Driven Multi-Agent Frameworks

Large Language Models (LLMs) have been widely applied in decision-making tasks recently. For one thing, some studies focus on the reasoning capabilities of LLMs. ReAct [87] investigates the generation of reasoning traces and actions, and ExpeL [91] delves into the realm of autonomous experiential learning. For another, increasing works aimed at enhancing the LLMs' interactive capabilities with external environments. Toolformer [65] enables LLMs to interact with APIs, whereas Palo et al. [56] explores their role within reinforcement learning agents. "Do As I Can, Not As I Say" [2] introduces a grounding mechanism, ProgPrompt [67] targets robotic task planning, and VisualGPT [12] examines multi-modal adaptability. Moreover, several works focus on multi-agent cooperation driven by LLMs. The frameworks proposed by Hong et al. [30], Du et al. [21], Liang et al. [52], Wei et al. [80] and Zhang et al. [89], explore various aspects ranging from collective decision-making to divergent thinking.

Prior work has shown the effectiveness of multi-agent frameworks in addressing complex tasks, which we have applied in designing Prompt2Task.

2.4 Incorporating User Feedback for Agent Improvement

Interactive machine learning (IML) [4, 22, 23] aims to achieve complementary benefits by integrating human perception and intelligence with the computational power of computers [18]. The framework has seen extensive application in fields requiring annotation [69], classification [7, 16], and intricate data analysis [11]. In line with the principles of IML, Prompt2Task employs an ensemble of agents that evolve through user feedback. The key problem is how to incorporate minimal, low-cost user intervention to achieve the maximum improvement in system performance.

3 PROBLEM OF UI TASK AUTOMATION

UI task automation is traditionally performed by mimicking predefined, rule-based operations on graphical user interfaces [44]. Recent advances in natural language processing and human-centric design have indicated a more dynamic, context-aware era in the task automation, guided by textual prompts. These textual inputs are not only easily formulated but also can offer valuable information, such as the task's goal, steps, related parameters, and specific applications.

In this section, we introduce Prompt2Task model to better understand the UI task automation process driven by text. Based on this model, we analyze the challenges within the task automation, which inform our system's architecture.

3.1 Prompt2Task Model

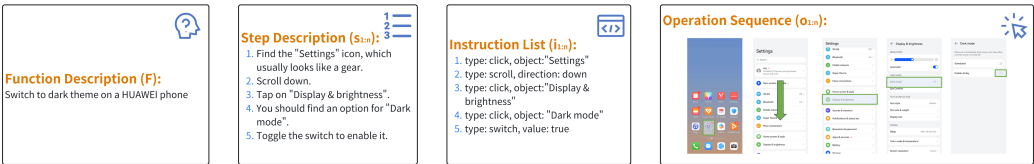


Fig. 2. Components of a Prompt2Task model.

Prior task models were developed to provide formalized descriptions of tasks, typically in the context of user interface design. They are primarily used to describe complex system interactions

and workflows, such as those involving concurrency and collaboration in high-complexity tasks [53]. To address task automation on mobile GUI, we propose a simplified UI task model based on ConcurTaskTrees [61], as shown in Figure 2. This model consists of the following core components:

- **Function Description (F):** A piece of natural language text that represents the overarching objective the automation task aims to achieve.
- **Step Description ($s_{1:n}$):** A list of textual items detailing individual steps within the automation process, each specified in natural language.
- **Instruction List ($i_{1:n}$):** A list of formalized instructions, derived from natural language text, encoded in a machine-friendly standard language to direct the operation sequence.
- **Operation Sequence ($o_{1:n}$):** An ordered set of operations required by the automation task, dynamically adjusted based on the textual prompts and the real-time operating environment.

In the Prompt2Task model, F and $s_{1:n}$ are used to express the task's goal and steps through natural language. While natural language lacks the rigor of prior task models such as the tree structure of ConcurTaskTrees [61] or the precise descriptions of concurrency, communication, and temporal order in LOTOS [8], it provides sufficient flexibility to describe the core requirements of a task effectively. More importantly, it reduces the complexity for users, making it more accessible for non-experts. Additionally, $i_{1:n}$ and $o_{1:n}$ in the Prompt2Task model are represented as linear lists, since the execution of each automation task is typically carried out on a single device through simulated operations, making task execution inherently linear. Even when task planning involves branching structures, the actual execution remains linear. It does not require the complex structure in Hierarchical Task Analysis [70] or concurrency mechanisms in Event-Driven Process Chains [64]. By generating the operation sequence dynamically, the model is flexible enough to adapt to real-time changes. Therefore, the Prompt2Task model is well-suited to handle the dynamic and diverse requirements of UI task automation efficiently.

3.2 Challenges for UI Task Automation

In this paper, we focus on the process of inputting a textual prompt P to generate the aforementioned Prompt2Task model and ultimately execute the automation task based on the model. The information contained in P is insufficient to accurately generate the model components. Therefore, the process necessitates the integration of additional knowledge, such as application-specific details, user context, and current GUI state. In this process, we identify the following four challenges:

1. Input: The input can be any text related to the task, ranging from formal manuals and tutorials to diverse user inputs. Users' expressions vary greatly, influenced by their differing levels of knowledge. Developers or skilled users can provide clear, standard instructions for specific operations, while novices or elderly users often struggle to articulate even basic functional requirements, sometimes only having a vague concept of the desired outcome [37]. Even skilled users might expect to automate tasks with minimal and simple inputs. Most current research [49, 57, 60] accepts explicit, step-by-step commands, such as "click the confirm button" or "input Hello in the edit box", but falls short in handling more complex commands like "activate developer mode", and are particularly challenged by ambiguous requests like "change the appearance".

2. Output: The output environment for task automation is dynamic, which means that the operation sequence ($o_{1:n}$) depends not only on the textual instructions ($i_{1:n}$) but also on the operating environment. Factors like smartphone model [3], app version [73], and page state [33] can all influence the operation sequence. For example, as shown in Figure 3, when a user wants to post a moment, varying page states may require different operations.

3. Error Handling: Successfully executing a multi-step automation task requires that each step be error-free, which is a strict condition. Besides the two challenges mentioned above, many

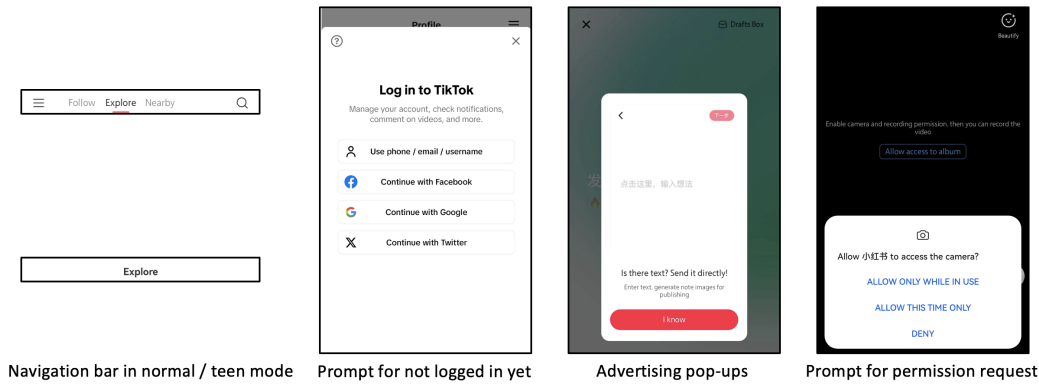


Fig. 3. For the same textual prompt “post a new moment”, however, the page state can affect the specific operations required.

Table 1. Comparison to prior work. The comparison dimensions include the type of input content, whether the range of supported tasks is limited, whether the operation sequence can be updated in real-time based on the actual UI state, how to handle errors, and what types of knowledge are incorporated.

Prior work	Input	Unlimited Tasks	Update	Error Handling	Knowledge
UIBert [5]	Single step	✓	×	ED&R	Prepared dataset
Spotlight [41]	Single step	✓	×	ED&R	Prepared dataset
Ferret-UI [51, 88]	Single step	✓	×	ED&R	Prepared dataset
MUG[43]	Single step	✓	×	UI	Prepared dataset, interaction history
AdbGPT [25]	Steps	✓	×	ED&R	LLM
Voicify [75]	Target Page+Steps	×	×	ED&R	Prepared dataset
UGIF [73], Macro [32]	Function	×	×	ED&R	Prepared dataset
AXNav [71]	Function	✓	✓	ED&R	LLM
AutoDroid [81]	Function	✓	✓	ED&R	Prepared UI graph
GPTVoiceTasker [74]	Function	✓	✓	UI	Expanding UI Graph
RTA [90]	Task-related texts	✓	✓	ED&R	LLM
CogAgent [31]	Task-related texts	✓	✓	ED&R	Fine-tuned LLM
VisionTasker [68]	Task-related texts	✓	✓	ED&R	LLM, prepared dataset
Mobile-Agent [79]	Task-related texts	✓	✓	ED&R	LLM
Prompt2Task	Task-related texts	✓	✓	UI	Online tutorials, accumulated knowledge

ED&R: Extend the dataset and retrain models to handle errors.

UI: User intervention.

1 Only pages with deep links are supported.

2 Only functions in the dataset are supported.

other factors can lead to errors, such as missing accessibility labels on GUIs [76, 84, 86], inadequate semantic recognition of GUI elements [19, 39, 54, 59], and missing parameters during execution [55]. These issues can result in unpredictable outcomes and significantly compromise the reliability of the task automation system. While we acknowledge that data-driven models have the potential to improve accuracy with sufficient training data in the future, we are more eager to optimize the model's performance in a controlled and targeted manner for current usage scenarios. Usually, a simple and minor correction is sufficient to ensure the success of a task [43].

4. Knowledge: Since the process requires the infusion of knowledge, ensuring reliable knowledge acquisition is critical. While LLMs are promising for planning and reasoning, they often lack the necessary domain-specific knowledge, leading to errors or hallucinations [36], especially with uncommon or deeply hidden functions in applications [74]. Additionally, LLMs are often inefficient [76], making the automation process even slower than manual execution, which users find frustrating [92]. Recent works have started to recognize the importance of domain-specific knowledge, such as continuously constructing UI graphs to enhance task planning [74]. We believe a more systematic and comprehensive application of domain-specific knowledge is needed—one that ensures the knowledge required for task automation is not only accurate but also efficiently extracted and applied, improving both reliability and speed throughout the entire process.

Considering the aforementioned challenges, prior work has proposed various solutions, each tackling some aspects of these issues to varying degrees. As shown in Table 1, we identify significant room for improvement in their practical effectiveness when applied in real-world scenarios. To this end, we have designed and implemented Prompt2Task, which provides targeted solutions to the challenges outlined above.

4 PROMPT2TASK

To address the above challenges and align with the components of the Prompt2Task model, we propose Prompt2Task, as shown in Figure 4. We have divided the process into three stages:

Information Collection: To tackle the challenge posed by inputs, Prompt2Task extracts useful information and underlying intentions from the textual prompt P . It supplements missing information in the user's expression through predictive analysis, asking the user or retrieving external knowledge. This stage prepares all relevant natural language information: the function description F and step description $s_{1:n}$.

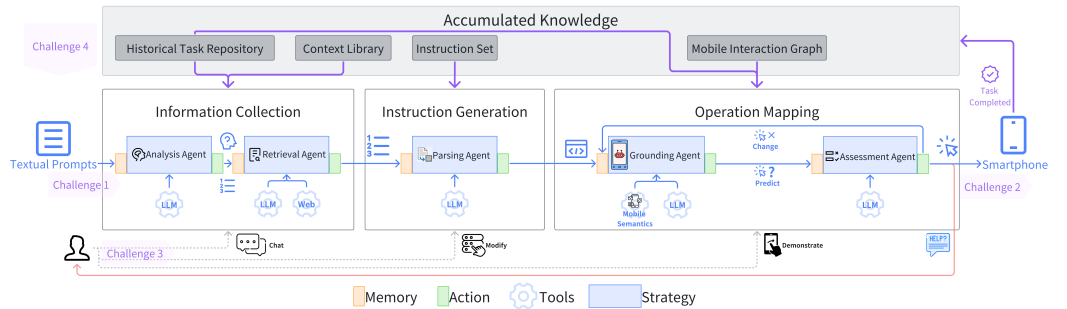


Fig. 4. System design of Prompt2Task. The blue solid lines represent the main flow of the task automation process, the grey dashed lines represent optional user intervention, the purple lines represent knowledge accumulation after task completion and its optimization to each stage, and the red solid line denotes instances where the agent seeks user assistance. Each stage's workflow details are shown in Figure 5, 7, 8.

Instruction Generation: Employing the collected information, this stage strategizes the task's progression. Given the instruction definition, Prompt2Task converts the step description into a list of standardized instructions $i_{1:n}$.

Operation Mapping: To address challenges from the dynamic output environment, such as rapidly updated pages, Prompt2Task should make adaptive decisions beyond pre-defined instructions. Specifically, it includes a deep understanding of the real-time environment, flexible grounding, and assessment to enhance the reliability of operations. This stage finalizes the generation of the operation sequence $o_{1:n}$.

To optimize the system in a controlled and targeted manner, Prompt2Task is designed to seek user assistance when errors or uncertainties arise, ensuring the task remains on track. The trade-off here is balancing user oversight to enhance system effectiveness while minimizing user burden. Prompt2Task achieves this by making the results of each stage transparent and requesting user help only when necessary. Even if users are not willing to intervene, Prompt2Task can autonomously complete the entire process.

In terms of knowledge, we incorporate online tutorials to expand the task range as open-ended as possible and enhance the reliability of planning. The accumulated knowledge from past tasks, including context and execution information, is processed and stored to improve Prompt2Task's future reasoning capabilities and efficiency.

Throughout the above three stages, different types of intelligence and knowledge are applied to achieve the corresponding goals. Therefore, we utilize a multi-agent system framework. Each agent, based on their assigned sub-tasks, is responsible for specific knowledge retrieval, tool utilization, decision-making, and action execution. These agents collaborate to complete the overall task. Figure 4 illustrates how Prompt2Task is partitioned into specialized agents and a unified knowledge base. We will delve into each in the following subsections.

4.1 Multi-Agent Design

Drawing inspiration from the prior agent system frameworks [20, 30, 58, 72], each agent within Prompt2Task is composed of several key components: memory, tools, strategy, and actions, which support the agent's fundamental functions. Additionally, each agent is equipped with a user interface that allows users to observe, modify, or augment agent decisions, as shown in Figure 4. The interface is not just for monitoring but aims to foster human-agent collaboration.

We have designed five distinct agents specifically tailored to the three stages of the process.

- **Information Collection** (Figure 5):
 - *Analysis Agent:* Extracts and analyzes information implicit in the textual prompt to construct a complete **function description** F .
 - *Retrieval Agent:* Acquires external knowledge relevant to the function description and builds an enriched **step description** $s_{1:n}$.
- **Instruction Generation** (Figure 7):
 - *Parsing Agent:* Transforms the collected information into a list of **instructions** $i_{1:n}$.
- **Operation Mapping** (Figure 8):
 - *Grounding Agent:* Predicts **operations** $o_{1:n}$ on the smartphone based on the generated instructions.
 - *Assessment Agent:* Reviews the predicted operations, evaluates if user intervention is required, and eventually executes operations on smartphones.

Subsequently, we will introduce how each agent works.

4.1.1 Analysis Agent. The agent aims to analyze and confirm task-related information from the textual prompts.

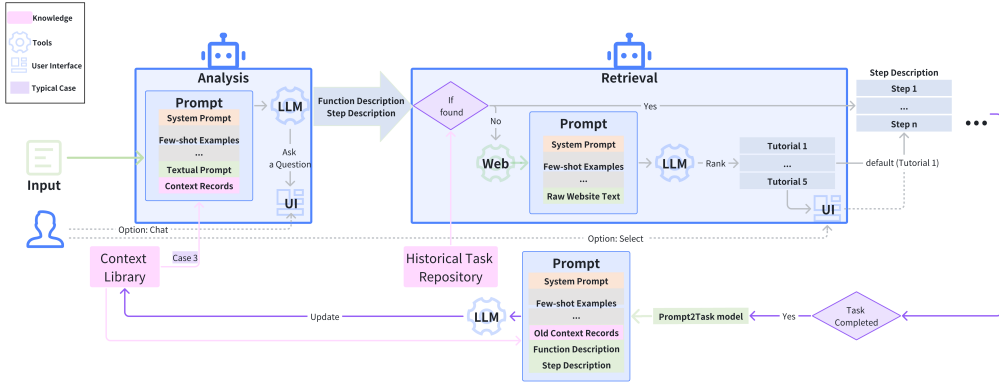


Fig. 5. Information collection workflow. Task-related functions and step descriptions are collected by the *Analysis Agent* and *Retrieval Agent*. The colors of the components within the prompt indicate their sources, which are the same color elements pointing to the prompt with the same color arrows. Gray arrows represent the normal task flow, while purple arrows indicate the accumulation of knowledge.

The prompts may include descriptions of the function or specific steps in the model, as outlined in Section 3.1. The function description must be confirmed and refined from the input. The original function descriptions are often informal and may lack essential context, such as specific application names or phone models. Figure 6 demonstrates how the agent interacts with users to refine these descriptions. Utilizing the LLM, the agent successively adopts three key strategies:

- **Normalization:** Users often describe the desired outcome without accurately naming the specific function. To address this, the agent rephrases the original input to reduce colloquial and inaccurate expressions.
- **Automated Context Enrichment:** The agent utilizes the LLM to enhance the user's original input by incorporating the user's record from the context library (detailed in Section 4.2.2). The record is directly added in the prompt to the LLM, enabling the generation of several potential candidates for the user to choose from.
- **Interactive Context Enrichment:** If the agent determines that relevant context information is still missing after the automated enrichment, it queries the user to obtain the missing details. The agent continues to chat with the user until it determines that the function description is sufficiently clear.

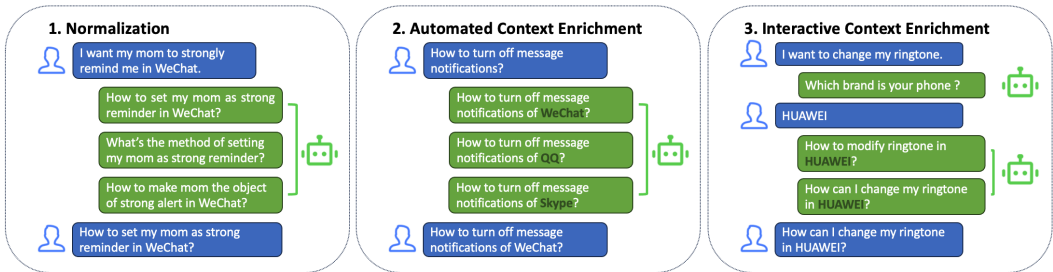


Fig. 6. Three scenarios where the agent interacts with the user to refine the function description.

After refining and enriching the function description, the user confirms it via chatting (Figure 11(a)). The agent then bundles the function description, along with step descriptions (if available), and forwards them to the *Retrieval Agent* for subsequent processing.

4.1.2 Retrieval Agent. The *Analysis Agent* can ensure a clear function description, but it may fall short when it comes to providing detailed step descriptions, which the prompt might not always cover. Addressing this void, the *Retrieval Agent* is responsible for preparing a complete step description. For efficiency, the agent prioritizes the following strategies:

- If the user confirms that the step description in the prompt is complete, the system advances to the next stage without additional information collection.
- Otherwise, the agent searches the historical task repository (detailed in Section 4.2.1) for matching the most relevant historical task model, using function description as the keyword. If a match is found, the agent will use the matched task model, which has been previously used within our system and is deemed reliable.

If the user is dissatisfied or no match is found, the agent searches for relevant tutorials on the Internet. The agent uses search engines, with the function description as a query. The tutorial texts from the fetched web pages are refined using SimpRead²'s rule library and summarized by the LLM. As illustrated in Equation 1, the score for each tutorial is calculated based on criteria that include their original search ranking, the data source, and textual quality assessed by the LLM [15]. Notably, the alignment with the original step description analyzed from the user's prompt is also a factor to consider, which is also assessed by the LLM. The constant parameters are empirically determined. Tutorials are ranked according to their score from highest to lowest.

$$\begin{aligned}
 \text{Score}(\text{Tutorial}_i) = & c_0 \times \text{originalRank}_i \\
 & + c_1 \times \text{weight}(\text{source}_i) \\
 & + c_2 \times \text{textQuality}_i \\
 & + c_3 \times \text{matched}(i, \text{oriStepDescription})
 \end{aligned} \tag{1}$$

As shown in Figure 11(b), Prompt2Task presents the top 5 highest-ranking tutorials to users, automatically selecting the highest-ranked tutorial in cases where users do not make a choice. By this stage, both the function and step description for the task automation are prepared in place.

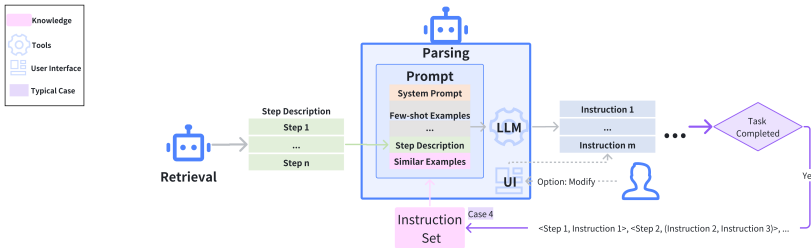


Fig. 7. Instruction generation workflow. Task-related instructions are generated by the *Parsing Agent*. The colors of the components within the prompt indicate their sources, which are the same color elements pointing to the prompt with the same color arrows. Gray arrows represent the normal task flow, while purple arrows indicate the accumulation of knowledge.

²<https://simpread.pro/>

4.1.4 Grounding Agent. As shown in Figure 8, the final stage aims to predict specific operations on the smartphone given the instructions. Specifically, according to the object description in the instruction, it identifies the UI widget on the smartphone and thus executes the relevant operation.

Initially, the agent filters out all widgets that lack both interactive attributes and text descriptions. Interactive attributes include being clickable, long clickable, editable, focusable, checkable, and scrollable. Text descriptions include both explicit text and implicit content descriptions. They are all available in the layout hierarchy. Then the agent relies on the mobile semantics module to bridge the gap between text-based instructions and GUI described by layout hierarchy and screenshots. Considering real-time efficiency, the module employs the *screen2words* model [77] to generate full-page textual descriptions and utilizes a combination of OCR and *pix2struct* [40] to describe widgets which lack text descriptions.

Subsequently, to predict the operation efficiently and effectively, the agent adopts a multi-layer strategy as follows:

Historical task invocation: If the given instructions are recurring, the agent consults the historical task repository (in Section 4.2.1) to identify widgets used in the past. It then compares these widgets with the ones on the current page, using heuristic rules that compare their rich GUI attributes. A widget is considered a match only if it exceeds the preset similarity thresholds.

New instruction-based grounding: For new instructions or when the historical task invocation fails, the agent employs the following strategies:

- (1) **Strict text matching:** The agent uses strict text matching to find a widget closely resembling the object description. A preset similarity threshold must be met to confirm a match. If this strategy fails, the agent then employs the LLM to decide which of the remaining five strategies to use: redirect, add, skip, expand, and block.

- (2) **Redirect:** The agent leverages LLM to predict the widget most suited to execute the given instruction. It focuses on semantic similarity with the operation and object description in the instruction.

However, instructions and operations do not always correspond one to one. To make the mapping more flexible, the agent can choose from the following exploratory strategies:

- (3) **Add:** The agent incorporates additional steps when necessary, often due to outdated or erroneous instructions, or when an instruction step requires expansion (Figure 9(a)). The agent leverages as much contextual information as possible to infer the most probable operation rather than solely relying on the initial instructions. When the agent predicts an incorrect operation, the *Assessment Agent* can help it retract and potentially find the correct operation. This strategy is crucial when there are unexpected changes on the page (Figure 3).
- (4) **Skip:** The agent skips unnecessary or irrelevant steps to better align the current page state with the instructions. This strategy helps in addressing inconsistent starting points, allowing the process to continue smoothly even if some steps are not applicable (Figure 9(b)).
- (5) **Expand:** When all instructions have been finished, or the remaining instructions are totally useless, the agent keeps trying to explore potential operations, until the *Assessment Agent* indicates task completion.
- (6) **Block:** The agent is currently unable to proceed as it cannot identify a suitable interactive widget, often due to the lack of information on the current page. In such cases, the agent requests user intervention by describing the current situation and the expected user actions. The user is asked to manually demonstrate the correct operation on the mobile GUI, as shown in Figure 11(d). If the user chooses to ignore, the agent will re-invoke the LLM to employ one of the above strategies.

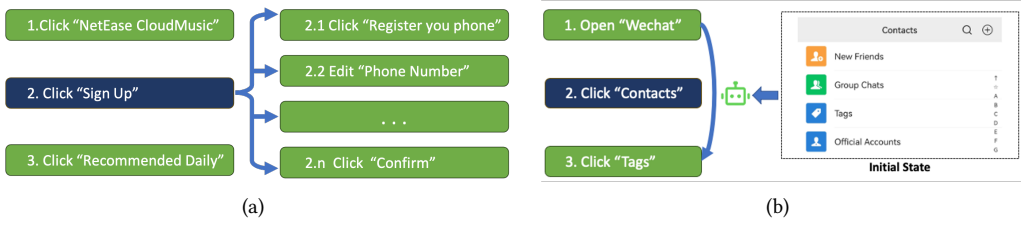


Fig. 9. (a) The compound instruction *Click "Sign up"* requires additional instructions. The agent can add the necessary steps. (b) The state doesn't match the first instruction. The agent can skip unnecessary instructions.

As the agent persistently accrues knowledge and insights, it improves mobile semantics extraction and LLM reasoning (further discussed in Section 4.2.4). For the implementation details of each strategy, see Appendix A.4.

4.1.5 Assessment Agent. The agent is introduced to proactively address potential pitfalls and simplify decision-making. It ensures the *Grounding Agent* remains focused on its primary tasks without being overloaded with error detection.

As shown in Figure 8, for efficiency, historical task invocations are thought to be sufficiently reliable and thus do not require assessment, allowing for direct execution. For new instruction-based grounding, the agent first assesses whether the current operation is a previously failed attempt, preventing endless loops during exploration. Subsequently, the agent evaluates whether the operations executed up to the present are reasonable. The agent leverages the LLM for reasoning. Based on contextual information including the current page description, operation history, and the current instruction, the agent adopts one of the following strategies:

- **Follow:** The agent determines that the operation is reasonable and proceeds with execution on the smartphone.
- **Change:** The agent determines that the operation does not meet the current task requirements and instructs the *Grounding Agent* to remove the erroneous option and make a new prediction.
- **Retract:** If the *Grounding Agent* has previously attempted erroneous expansions leading to irrelevant pages, the agent performs a global back operation to return to the previous step.
- **Finish:** Usually, tasks are not completed simply by executing all instructions, due to outdated tutorials or errors in preceding stages. Therefore, the *Grounding Agent* will automatically choose the "expand" strategy to identify further necessary operations after all instructions have been executed. Based on the contextual information, the *Assessment Agent* determines whether the task is actually completed. The execution process only terminates when the *Assessment Agent* confirms task completion.

However, sometimes the agent is not sufficiently confident in adopting any of the above strategies. In such cases, we consider it appropriate to seek user assistance. Therefore, the agent employs the LLM to assess if it is confident in its decision. If the agent is unconfident, it describes its current decision and prompts the user to demonstrate the correct operation. Figure 10 illustrates a scenario where the agent prompts user intervention due to low confidence for decision. Similarly, Figure 1 showcases a situation where the instruction "Click a ringtone you like" introduces an ambiguity that necessitates user assistance. More details are in Appendix A.5.

Notably, even if the agent prompts to seek user assistance, the user has the option to ignore the prompt, in which case the agent will directly execute according to its own decision.

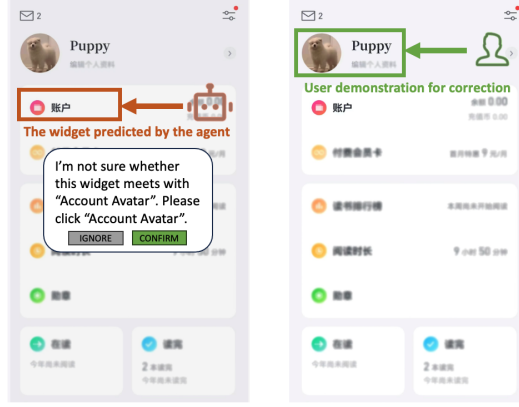


Fig. 10. A case of decision uncertainty prompting user intervention. The target object is described as “Account Avatar”. The agent, which relies heavily on textual descriptions for predictions, inaccurately predicts “account” because the avatar widget lacks a precise textual description. Due to low confidence in its prediction, the agent seeks user intervention.

4.2 Accumulated Knowledge

As shown in Figure 5, 7, 8, the agents draw from an accumulated knowledge base, which is continually improved through user usage and intervention. As a feedback mechanism, the knowledge base facilitates the learning of the agents, thereby enhancing the entire system’s efficiency and task success rates.

According to the above agents’ knowledge requirements, Prompt2Task requires four types of knowledge:

- *Historical Task Repository* stores successfully executed automation tasks, which typically exhibit a higher success rate in future runs. Additionally, it retains adjustments tailored to specific operating environments or updates to previously outdated tutorials, eliminating the need for manual reconfigurations upon reuse.
- *Context Library* stores and continuously updates user-specific mobile parameters. It helps the system supplement necessary details when faced with incomplete user expressions.
- *Instruction Set* focuses on improving the system’s ability to extract instructive information from text. Similar examples become valuable learning materials.
- *Mobile Interaction Graph* strengthens the system’s understanding of app functionalities and navigation logic. It serves as an auxiliary routing tool, helping agents to better predict the operations.

We will introduce them in details and explain how they bring improvements to the agents.

4.2.1 Historical Task Repository. Upon successful execution, Prompt2Task archives the Prompt2Task model’s data, according to the definition in Section 3.1. Since the *Grounding Agent* may not always align operations with the original instructions, the repository uses the sequence of operations as the ground truth. These operations are then reverse-engineered into corresponding instructions and step descriptions via simple mapping rules, as detailed in Appendix B.1.

The historical task repository provides several optimizations to the agents:

- The *Retrieval Agent* will prioritize historical task models, enhancing data reliability and avoiding the time on retrieving online tutorials whenever possible. The agent identifies the most similar historical task models based on the current function description. Only when

the similarity exceeds the preset empirical threshold is the model considered a match and directly used for subsequent processing.

- The *Grounding Agent* gains enhanced predictive capabilities. For instructions stored in the repository, it can now identify corresponding UI widgets through a richer feature set that incorporates layout hierarchies and screenshots, rather than relying solely on text descriptions from instructions (as exemplified in Case 1 in Appendix B.3). As long as the features of the UI widgets do not change significantly, the heuristic rules employed by the agent can accurately and quickly identify the widgets and trigger the appropriate operations. Additionally, the stored mapping between instructions and operations serves as a learning resource for the agent. If the historical task invocation fails, the agent finds the historical instruction most similar to the current one and then incorporates the corresponding mapping into the prompt, thereby aiding the LLM in decision-making (for details, refer to the prompt engineering of the *Grounding Agent* in Appendix A.4).
- The mobile semantics module in the *Grounding Agent* benefits from a richer set of textual descriptions for widgets. For the widgets' screenshots stored in the repository, the module computes their embeddings [77]. When new images with similar embeddings to the stored ones appear, the module uses the stored textual descriptions for them, which are more accurate. It is particularly effective for recognizing icons; as long as the icons' images maintain a similarity that does not exceed an empirical threshold, the agent can accurately label them when they are encountered again (as exemplified in Case 2 in Appendix B.3).

4.2.2 Context Library. Derived from a formative study involving 24 college students and 20,265 online Q&A questions (detailed in Appendix B.2), the library classifies these parameters into three categories based on acquisition methods:

- (1) **Automated System Capture:** Parameters that are available from the mobile GUI, like battery status or current application in focus, are automatically collected by the system. Sensitive data is not stored.
- (2) **User Input:** Device-specific information authorized or provided by the user is also collected. It could include permissions granted (e.g., access to camera or location), as well as other profile details (e.g., preferred language) which users can actively input through forms presented by the system.
- (3) **User-Driven Updates:** The library continuously updates based on user-system interactions, primarily involving parameters mentioned by users in historical tasks, such as connected devices or Wi-Fi names.

In the library, a unique set of parameter records is maintained for each user (refer to the example in Appendix B.2). To support user-driven updates, after each completed automation task, the original records and the current task model are provided to the LLM to generate the latest records.

The *Analysis Agent* accesses the library to find the user's historical records and incorporates these records as part of the prompt (for details, refer to the prompt engineering of the *Analysis Agent* in Appendix A.1). With the aid of the library, missing parameters for recurring textual prompts are automatically supplemented, which users have the option to confirm. For instance, the earphones mentioned by the user in a certain use (refer to Case 3 in Appendix B.3) will be collected into the library and automatically suggested to the user in future tasks. Additionally, if the user's context changes and the parameters saved in the context library become outdated, users can correct them while chatting with the *Analysis Agent*, and the parameters in the context library will be updated after the completion of the new task.

4.2.3 Instruction Set. The *Parsing Agent* is tasked with converting natural language step descriptions into formalized instructions. While conceptually straightforward, practical application introduces issues, including the emergence of new operation, parameter, or object types and synonyms or deviation from the format in Table 2. To address these issues, the instruction set is regularly updated to include misinterpreted cases and their corrections. These cases can be resolved by user manual corrections, but most are resolved by instructions reverse-engineered from successful automation tasks, as described in Section 4.2.1. Similar to the historical task repository, the instruction set stores the mapping between the description of a single step and the instruction it is parsed into. For each step that needs parsing, the agent identifies the most similar description from the set, and incorporates the historical mappings into the prompt for the LLM, thereby enhancing the LLM’s parsing capabilities.

Another important issue is related to compound instructions (as exemplified in Figure 9(a) and Case 4 in Appendix B.3). For such cases, the user marks the compound instruction and its decomposed result, which are then recorded in the instruction set. Hence, if a compound instruction reoccurs, the *Parsing Agent* will automatically decompose it.

4.2.4 Mobile Interaction Graph. In the graph, each node represents a visited mobile page, and edges link interactive widgets to the pages they lead to [17]. As the system navigates more pages, the graph evolves dynamically. Specifically, for a completed automation task, the operation sequence naturally corresponds to a series of page transitions. To represent these transitions in the graph, Prompt2Task uses a heuristic rule-based similarity function [33] that considers the texts and their locations on pages, to calculate their similarity and categorize the pages. The similarity thresholds have been established using Receiver Operating Characteristic (ROC) curve analysis. Similar pages share the same node in the graph, and edges between these nodes represent the operation objects that trigger transitions between pages.

The graph enables virtual validation of each operation before actually executing on a smartphone. If the edge already exists in the graph, it anticipates the future pages that could be visited after the operation, thus enriching the contextual understanding for the *Grounding Agent* and the *Assessment Agent* (see Appendix A.4 and A.5 for prompt details). As long as the page transition relationships remain unchanged, the graph can provide the agents with accurate information about future pages (as exemplified in Case 5 in Appendix B.3).

4.3 Implementation

Prompt2Task is divided into two primary components: a server and a client. The server handles information collection, instruction generation, and knowledge accumulation. It is developed using the Flask framework and crawls search results from Baidu, Bing, and Google by simulating web browser actions. The client is an Android application installed on smartphones, designed to interact with users and execute automation tasks. As illustrated in Figure 11, the client provides a GUI for user input, selection, modification, and demonstration. During task execution, it runs in the background on the smartphone and utilizes the Accessibility Service API ³ to access page layout data, and the MediaProjection API ⁴ to capture screenshots. The client triggers operations on the smartphone through the Accessibility Service API. When user demonstration is required, an overlay is provided to monitor the user’s demonstrations. Actions captured by this overlay are recorded by the system and then transferred to the native mobile GUI to trigger the corresponding operations. Prompt2Task’s agents utilize the GPT-4 ⁵ API for their large language model needs. All involved

³<https://developer.android.com/reference/android/accessibilityservice/AccessibilityService>

⁴<https://developer.android.com/reference/android/media/projection/MediaProjection>

⁵<https://platform.openai.com/docs/models/gpt-4>

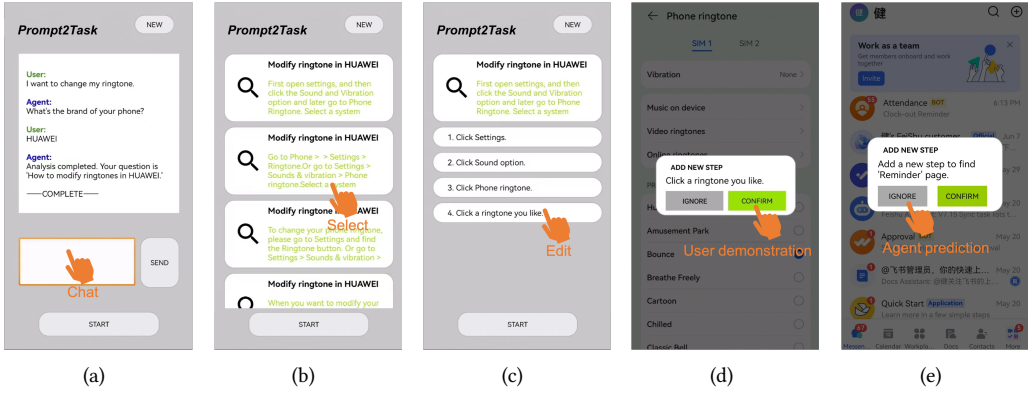


Fig. 11. Runtime screenshots of the Prompt2Task client. (a) The user chats with the *Analysis Agent* to finalize the function description. (b) The *Retrieval Agent* provides 5 tutorials for the user to choose from. (c) The *Parsing Agent* generates instructions, which the user can edit. In (a) (b) (c), if the user prefers not to intervene, they can click “START” to skip. (d) The *Grounding Agent* fails to find the target widget and requests the user to provide a demonstration. The user can begin the demonstration by selecting “CONFIRM”. (e) The *Assessment Agent* is unconfident in the current predicted operation and requests user intervention, but the user can still choose “IGNORE”, allowing the agent to proceed with its prediction.

prompts are detailed in the appendix. In tasks involving textual similarity, including the *Retrieval Agent* finding the most relevant historical task model, the *Parsing Agent* finding the most similar step descriptions, and the *Grounding Agent* finding the most similar instructions, we use LaBSE [24], which is a state-of-the-art multilingual sentence embedding model, offers an effective trade-off between efficiency and accuracy.

5 PERFORMANCE EVALUATION

This section assesses the performance of Prompt2Task’s three-stage pipeline—information collection, instruction generation, and operation mapping—as well as the overall system performance.

5.1 Experiment Setup

Prompt2Task is designed to fulfill a wide range of real-world needs through textual prompts provided by users, which involve various applications and built-in functionalities. Therefore, we specifically created a dataset for the experiment.

Targeting Chinese speakers, we selected 10 Chinese applications (9 applications and 1 system) for evaluation, as shown in Table 3. Five of them are top-ranked in terms of active users as of 2021, and four have rankings beyond 100. The diverse selection enables us to evaluate Prompt2Task’s performance across different user activity levels and application complexities. We constructed 100 ground-truth Prompt2Task models by the following steps:

- We collected 37,987 questions related to these applications by crawling a recognized online Q&A platform⁶. The questions were sorted based on their view counts. The top half were categorized as common, while the bottom half were categorized as uncommon. We manually selected 5 common and 5 uncommon questions for each application, ensuring all 10×10

⁶<https://jingyan.baidu.com/>

Table 3. Applications and tasks overview in the experiment. WeChat, QQ, TikTok, Beautiful Weather, Safe and Sound, and Film Encyclopedia were also selected for the user evaluation in Section 6.

Application	Category	Activity Ranking	Step Range	Task Examples
WeChat	Message	1	4~10	1. View WeChat Bean details 2. Clear cache
QQ	Message	2	3~8	1. Share QQ screen 2. Provide feedback on an issue
TikTok	Social Media	3	3~10	1. Cancel the monthly payment 2. View TikTok shop
Alipay	Payment	4	3~8	1. Set the notification sound 2. View Alipay points
Weibo	Social Media	5	4~7	1. Check creation earnings 2. View corrected followers
Lark	Office	173	5~10	1. Log out 2. Add Lark reminders to the workspace
Beautiful Weather	Weather	366	3~7	1. Update the weather every 3 hours 2. Observe the weather in Beijing
Safe and Sound	Lifestyle	392	3~8	1. Set the shipping address 2. Enable the night mode
Film Encyclopedia	Entertainment	671	4~10	1. Join the user experience program 2. Bind QQ with the Film Encyclopedia
System Tools	System	-	4~8	1. Turn off the pure mode 2. Hide photos

questions were unique. Each question was used to construct one Prompt2Task model and each was manually annotated with a ground-truth function description.

- For each model, we manually identified the highest-quality tutorial on tutorial websites based on the given function description. This tutorial text serves as the ground-truth step description for the model.
- We utilized crowdsourcing to collect textual prompts that participants would use to generate automation tasks. The tasks were hosted on a public crowdsourcing website we developed, where participants were required to formulate prompts based on the ground-truth function descriptions provided to them. Each participant was instructed to ensure the uniqueness and relevance of their submissions. After filtering out duplicate, low-quality, or irrelevant prompts, we collected 25 different textual prompts for each automation task, for a total of 25×100 prompts. Among them, 100 prompts contained complete step descriptions, without the need for further tutorial retrieval.
- According to the step description, we then manually annotated the ground-truth instructions for each Prompt2Task model, totaling 533 instructions.
- Finally, the experiment was conducted on an HONOR V20. For each Prompt2Task model, we manually recorded the ground-truth operation sequence on this device, totaling 543 operations. Notably, these operations do not directly correspond to the ground-truth instructions, often due to outdated tutorials. Details are available in our public dataset.

5.2 Experiment Method

In our experiment, we evaluated three configurations of Prompt2Task:

- Standard Prompt2Task.
- **Prompt2Task(AK)**: Prompt2Task enhanced with accumulated knowledge.
- **Prompt2Task(AK+minimal UI)**: Prompt2Task enhanced with accumulated knowledge and minimal necessary user intervention. Only essential user interventions during the **operation mapping** stage to provide important parameters are permitted. For instance, “Click a ringtone you like” in Figure 1 does require user intervention, while the scenario in Figure 10 does not inherently require it but arises due to the agent’s limitations.
- **Prompt2Task(AK+UI)**: Prompt2Task further enhanced with both accumulated knowledge and user intervention, providing perfect user intervention to maximize success rate.

The experiment assessed Prompt2Task’s performance at individual stages and across the entire pipeline. Notably, in a single stage where a task was not fully completed, without user intervention, the knowledge that Prompt2Task could accumulate was quite limited. Therefore, **Prompt2Task(AK)** was evaluated solely on its entire pipeline performance. When evaluating the performance of individual stages, we used ground-truth inputs, ensuring that any output deviation stemmed exclusively from the current stage, eliminating errors from preceding stages. Specifically, in the instruction generation stage, we input 100 ground-truth function and step descriptions; for the operation mapping stage, we input 100 ground-truth instruction lists. In evaluating the information collection stage and the entire pipeline, the input comprised 2,500 original textual prompts. To minimize the impact of input order and the inherent variability of LLM outputs, we randomly shuffled the inputs and conducted five rounds of testing to calculate the average performance.

We selected appropriate baselines for comparison at both individual stages and the entire process. For the information collection stage, manual queries on Google, Bing, and Baidu served as the baseline. For instruction generation, we used Seq2act [49] as the baseline. For the operation mapping stage, we employed strict text matching as the baseline. In assessing the entire pipeline, our baseline was using GPT-4 to generate instructions, followed by strict text matching for operation mapping. Additionally, considering some works that use step descriptions as input, we used LLM4Mobile [76] and AdbGPT [25] as baselines for the last two stages combined.

5.3 Experiment Results

The experiment results are organized based on the three stages within Prompt2Task as well as the performance of the entire pipeline.

5.3.1 Information Collection. This stage focuses on evaluating whether Prompt2Task can successfully generate a function description and step description based on a user-provided textual prompt.

Among the 2,500 textual prompts, 100 contained complete step information and didn’t require further online retrieval. Users accepted the analyzed results for 97.00% of these prompts, with a few exceptions due to the LLM’s data fabrication issues. The remaining 2,400 prompts were used to assess the top 5 tutorials retrieved by the system. The criteria for a “hit” was whether users agreed that a tutorial matched the ground-truth function description, regardless of on-phone executability. A tutorial at the top indicated that the system could automatically select it, eliminating the need for user intervention. Among the 2,400 prompts, 12.21% have incomplete descriptions of functions and required further supplementation.

Before the experiment, we benchmarked the search capabilities of Google, Bing, and Baidu using 100 ground-truth function descriptions. As Table 4 shows, Baidu outperformed the others, achieving

Table 4. Performance of various search engines in retrieving tutorials, using ground-truth function descriptions as input.

	Top-5 Hit Rate(%)	Average Rank	Top-1 Hit Rate(%)
Google	93.00	1.23 (sd=0.59)	79.00
Bing	96.00	1.25 (sd=0.70)	81.00
Baidu	100.00	1.01 (sd=0.10)	99.00

Table 5. Performance of information collection, using 2,400 textual prompts as input.

	Tutorials Match Function Description			Step Description
	Top-5 Hit Rate(%)	Average Rank	Top-1 Hit Rate(%)	Accuracy (%)
Manual(Google)	86.71	1.55 (sd=0.85)	64.92	-
Manual(Bing)	89.17	1.36 (sd=0.80)	69.38	-
Manual(Baidu)	97.21	1.42 (sd=0.76)	87.96	-
Prompt2Task	98.83	1.27 (sd=0.91)	89.38	88.38
Prompt2Task(AK+UI)	100.00	1.22 (sd=0.74)	90.83	99.21

a 100.00% Top-5 hit rate and a 99.00% Top-1 rate. The results demonstrate that all tasks could be matched with online tutorials, provided the descriptions are sufficiently standard and accurate.

However, prompts provided by users often lack accuracy, leading to a decrease in effectiveness when using the original prompts for manual searches on these websites. Using them as baselines, we tested the performance of Prompt2Task. As shown in Table 5, Prompt2Task improved the Top-5 hit rate from 97.21% (Baidu) to 100%, while achieving a Top-1 hit rate of 90.83%. On average, each prompt required 0.15 user interventions (sd = 0.52), which represents the number of dialogue turns between the user and Prompt2Task (excluding the initial input). Remarkably, without user interventions, Prompt2Task demonstrated a Top-1 hit rate of 89.38%, surpassing the baseline performance.

After the agent matches function descriptions with online tutorials, it extracts concise step descriptions from complex web data. The accuracy of step descriptions refers to the correct extraction from web pages, ensuring they are free from fabrication, omissions, or errors. When Prompt2Task automatically selected the first candidate, the accuracy was 88.38%. When users could select from five candidates, the accuracy rose to 99.21%.

5.3.2 Instruction Generation. This stage evaluates Prompt2Task’s ability to transform step descriptions into a list of instructions in the predefined format.

Using the ground-truth step descriptions of the selected 100 Prompt2Task models as input, we evaluated correctness using two metrics, focusing solely on textual interpretation:

- **Complete Accuracy:** The percentage of Prompt2Task models whose all generated instructions match the ground truth.
- **Partial Accuracy:** The proportion of correct instructions relative to the total ground-truth instructions.

We used the Seq2Act model [49], a transformer trained using the ANDROIDHOWTO dataset, as our baseline (after translating step descriptions into English). Table 6 reveals that Prompt2Task

Table 6. Performance of instruction generation.

	Partial Accuracy(%)	Complete Accuracy(%)	User Intervention
Seq2act [49]	70.13	54.00	-
Prompt2Task	84.54	73.00	-
Prompt2Task(AK+UI)	100.00	100.00	0.93 (sd = 1.34)

Table 7. Performance of operation mapping. Prediction accuracy is measured as the proportion of correctly executed operations out of the total number of operations, where an operation is considered correct if it is appropriate in the context at that time. The success rate is determined by the ratio of completed tasks.

	Prediction Accuracy(%)	Success Rate(%)	User Intervention
Text only	61.49	44.00	-
Prompt2Task	80.25	62.00	-
Prompt2Task(AK+minimal UI)	88.06	91.00	0.61 (sd = 0.77)
Prompt2Task(AK+UI)	92.11	96.00	0.68 (sd = 0.85)

achieved 100% accuracy for both complete and partial metrics. On average, achieving perfection required 0.93 user interventions per task (sd=1.34). It's worth noting that while perfection necessitated user interventions, the subsequent pipeline stages are designed to accommodate minor errors, reducing the need for strict user oversight. Without user intervention, Prompt2Task attained 84.54% partial and 73.00% complete accuracy. In cold-start scenarios, Prompt2Task has achieved performance comparable to that achieved by related works [49, 75], which require substantial training data (at least 10K-level).

5.3.3 Operation Mapping. This stage evaluates Prompt2Task's ability to execute operations on smartphones, guided by instructions. We used the ground-truth instruction lists of the 100 Prompt2Task models as input.

Our baseline for comparison utilized strict text matching to identify operation widgets. Table 7 shows the results. With user intervention (i.e., users can demonstrate any operations), the success rate reached 96.00%. The remaining 4% of tasks were unfeasible under the current test environment. On average, each task required 0.68 user interventions (sd = 0.85). Even without user assistance, the success rate was 62.00%, far surpassing the baseline. When users were allowed to provide necessary parameters, the success rate was 91.00%, indicating that 29.00% of the tasks require user-provided information even under optimal conditions.

In terms of the detailed performance without user intervention, Table 8 presents the frequency and accuracy of the *Grounding Agent*'s strategies when handling new instructions. The agent seldom uses the "expand" strategy, which also shows the lowest accuracy. This indicates that even though some instructions are wrong or outdated, they still hold great reference value. In most cases, the agent prefers to use "redirect" or "add" strategies to adapt to the current operating environment. Table 9 shows the frequency and accuracy of the *Assessment Agent*'s strategies for the operations predicted by the *Grounding Agent*. The agent lacked confidence in 16.53% of the decisions, of which 68.57% required necessary user intervention to complete the task. In the remaining cases, while user intervention is optional, it can help the agent make more informed decisions.

Table 8. Frequency and accuracy of the *Grounding Agent*’s strategies. The “block” strategy indicates the agent could not predict the operation, thus it has no accuracy measure.

	Frequency(%)	Accuracy(%)
Strict Text Matching	26.44	100.00
Redirect	30.17	90.72
Add	39.50	56.69
Skip	1.24	100.00
Expand	1.40	33.33
Block	1.24	-

Table 9. Frequency and accuracy of the *Assessment Agent*’s strategies.

	Frequency(%)	Accuracy(%)
Follow	81.89	83.46
Change	5.04	53.13
Retract	3.31	95.24
Finish	9.76	100.00

5.3.4 The Entire Pipeline. To evaluate the overall effectiveness of Prompt2Task, we input 2,500 textual prompts into the pipeline, which comprises **information collection**, **instruction generation**, and **operation mapping**.

Given the task-related textual inputs, we did not use UGIF [73] and CogAgent [31] as baselines, as it would be unreasonable to require them to support our tasks without pre-provided tutorial sets or fine-tuning data. Additionally, we did not use Voicify [75], as our verification showed that only 15% of the tasks’ target pages had intent filters configured in the app code, which means that it is impossible to reach the remaining task pages using only page descriptions. Therefore, we established our baseline using GPT-4 for direct instruction generation from user inputs, coupled with strict text matching for operations.

As shown in Table 10, the success rate for each stage is strictly determined by the complete accuracy achieved up to that stage. Importantly, the overall success rate is not a simple product of success rates at each stage, due to the system’s built-in flexibility for error correction in downstream stages. For instance, the operation mapping stage doesn’t strictly follow the generated instructions; it also utilizes the agent’s exploratory strategies. For **Prompt2Task(AK+minimal UI)**, the average number of user interventions per task is 0.42 (sd = 1.32), excluding the initial user input. For **Prompt2Task(AK+UI)**, the average number is 0.69 (sd = 1.31). Notably, this number is smaller than the sum of user interventions required for each stage as reported earlier, which shows that the accumulation of knowledge from completed tasks can reduce user interventions.

Additionally, we compared Prompt2Task with baselines that support only step description inputs. We tested the performance using ground-truth step descriptions as inputs, which eliminates errors in online tutorial retrieval. Notably, it does not guarantee that the ground-truth tutorial is entirely feasible in the current test environment. While 31% of the tasks involve outdated step descriptions, only 11.82% of the individual steps are outdated, which means that a significant portion of the steps are still correct, and only minor corrections may be needed to complete the tasks. As shown in Table 11, Prompt2Task handles outdated tutorials effectively, achieving a 96% success rate when user intervention is applied. The discussion on outdated tutorials is detailed in Section 7.1. Moreover,

Table 10. Success rate (%) across various stages, using 2,500 textual prompts as input.

	Information Collection	Instruction Generation	Operation Mapping
GPT-4 only, Text only	-	50.88	22.28
Prompt2Task	88.42	64.92	60.80
Prompt2Task(AK)	92.26	72.32	78.16
Prompt2Task(AK+minimal UI)	93.16	73.36	90.68
Prompt2Task(AK+UI)	99.12	99.12	95.24

Table 11. The performance of the entire system with ground-truth step descriptions as input. The definitions of prediction accuracy and success rate are the same as those in Table 7. Relative efficiency is the ratio of ground-truth operations to actual operations performed.

	Prediction Accuracy(%)	Success Rate(%)	Relative Efficiency(%)
LLM4Mobile [76]	54.58	23.00	89.68
AdbGPT [25]	65.97	36.00	87.86
Prompt2Task	80.15	62.00	85.03
Prompt2Task(AK+minimal UI)	87.78	91.00	76.23
Prompt2Task(AK+UI)	91.46	96.00	76.13

the prediction accuracy of **Prompt2Task(AK+minimal UI)** and **Prompt2Task(AK+UI)** is lower than the success rate because the system executed redundant exploratory steps, which also led to a decrease in relative efficiency. However, for the goal of fully completing the task, this trade-off is considered acceptable.

5.4 Error Analysis

We analyzed the errors that occurred in the three stages of Prompt2Task.

5.4.1 Information Collection. Prompt2Task effectively matches tutorial web pages but falls short in content extraction. Two main issues affect the performance. First, web pages containing excessively long text compromise the LLM’s extraction capabilities, leading to information loss and increased risk of generating inaccurate content. Second, the LLM also tends to generate fabricated step descriptions when interpreting the textual prompts.

To address these challenges, we have considered two options for improving text quality. Initially, we considered implementing a specialized assessment agent, similar to the one used in the **operation mapping**, to manage text quality. However, we ultimately chose a human-in-the-loop approach, considering the efficiency of manual text assessment and the extra time needed for an agent-based solution. Our current system allows users to manually select high-quality step descriptions for subsequent stages.

5.4.2 Instruction Generation. Prompt2Task’s automatic instruction generation achieves a partial accuracy of 84.54%, with errors distributed across six main categories as shown in Figure 12. These errors are generally resolved through simple user corrections. Specifically, 43.93% of errors are due to instructions that go beyond the existing definition (Table 2). Within this subset, 24.51% involve the LLM using near-synonyms (e.g., “drag down” instead of “scroll down”). The remaining 19.42% pertain to complex operations like “log in” or “register” that the LLM struggles to parse, which can be potentially resolved by the exploration in the subsequent stage. Additionally, “Content

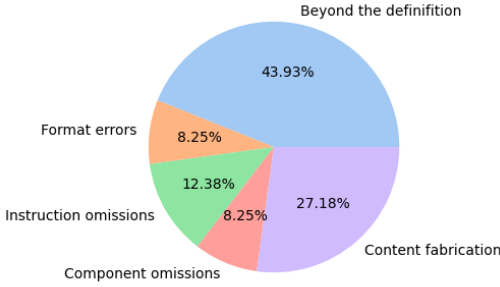


Fig. 12. Errors of instruction generation.

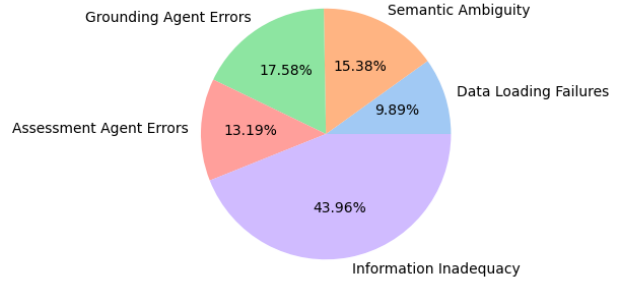


Fig. 13. Errors of operation mapping.

fabrication” represents the second largest category of errors, which is a notable issue of the LLM. It occasionally does not strictly adhere to the given step description and instead invents or elaborates on instructions.

Notably, not all errors necessitate user intervention. Many issues are addressed in the subsequent stage, improving the overall success rate compared to the instruction generation stage (Table 10). It is due to the system’s ability to self-correct certain operations. Essentially, generated instructions act as guidelines rather than absolutes, and even incorrect instructions offer valuable context that enhances the system’s overall performance.

5.4.3 Operation Mapping. Prompt2Task achieves an 88.06% prediction accuracy with minimal necessary user interventions. As detailed in Figure 13, errors can be categorized as follows:

Data Loading Failures: 9.89% are caused by issues in page data loading, making the system fail to access GUI widgets.

Semantic Ambiguity: 15.38% of errors arise from the system misidentifying widgets due to semantic ambiguities and fuzzy matching challenges. In this case, the agent can iterate attempts and refine the operation sequence based on triggered results.

Agent-Related Errors: 17.58% of errors are due to the *Grounding Agent* taking an unrecoverable erroneous direction in task execution. 13.19% of errors happen when the *Assessment Agent* wrongly rejects operations initially predicted by the *Grounding Agent*. Such errors occur when contextual information is insufficient, but they diminish as the *mobile interaction graph* expands. The graph can provide the agents with a richer context for each operation’s implications.

Information Inadequacy: Errors due to incomplete information constitute the largest proportion, accounting for 43.96%. Widgets without textual descriptions are particularly affected. Although the mobile semantics module in the *Grounding Agent* employs advanced image-to-text models, their ability to recognize the fine-grained semantics of widgets is limited. Fortunately, users can help interpret the widgets by demonstrating how the instructions map to the corresponding widgets. User demonstration can also provide context-specific interpretations, such as clarifying that “select” in a certain context means “switch” rather than “click”.

6 USER EVALUATION

Given the influence of user intervention on system performance, we assessed Prompt2Task’s usability through engagement with diverse users.

6.1 Experiment Setup

Participant Demographics: We recruited a total of 12 participants for our study through online postings in local communities. Each participant was compensated at a rate of \$15 per hour for their contribution. Before the study began, each participant was required to complete an informed consent form that detailed the nature of the study, the extent of data collection, and the installation of the Prompt2Task client on their personal devices (detailed in Section 4.3). We confirmed that no personal data would be retained post-study without explicit permission. This study has been reviewed and approved by the appropriate Institutional Review Board (IRB).

The participant's proficiency was determined through a pre-study questionnaire that evaluated their comfort level and frequency with smartphone usage. The participants were divided into the following two categories according to their proficiency.

- *Skilled Users:* 6 participants were proficient smartphone users (2 females; aged from 18 to 40, $M = 28.83$), who were familiar with a wide range of applications and functions.
- *Unskilled Users:* 6 participants were less proficient in smartphone usage (4 females; aged from 50 to 69, $M = 60.00$). They are elders, beginners, or infrequent smartphone users.

App Selection: The study included tasks across 6 applications—3 commonly used (WeChat, QQ, TikTok) and 3 less commonly used (Beautiful Weather, Safe and Sound, Film Encyclopedia). The selection criteria for these applications are consistent with those described in Section 5.1.

Task Selection: To ensure balanced task assignments, the participants were grouped into 3 sets, each containing 4 participants (2 skilled and 2 unskilled users). There are two types of tasks.

- *Predefined Tasks:* The 4 participants within a set are assigned the same 8 predefined tasks, split across 2 applications: one commonly used and one less commonly used. For each application, 2 common and 2 uncommon automation tasks are chosen, based on criteria detailed in Section 5.1. Part of the tasks are shown in Table 3. In each four-participant set, we assume that 'a' and 'b' are skilled users, whereas 'c' and 'd' are unskilled users. To ensure balance, for each application, one common task is performed manually by 'a' and 'c', while 'b' and 'd' use Prompt2Task. For the other common task, roles are reversed: 'a' and 'c' use Prompt2Task, while 'b' and 'd' perform it manually. The approach is similar for uncommon tasks.
- *User-Suggested Tasks:* In addition to the predefined tasks, each participant is allowed to propose 2 automation tasks for each of their assigned applications. We encouraged participants to suggest challenging tasks. For these tasks, participants will employ both manual execution and Prompt2Task. To counterbalance the effects of the method sequence, the order of methods is alternated across tasks.

Methods: Almost all tasks in the experiment, requiring in-depth application functionality, have been validated as unsupported by Google Assistant and HUAWEI Celia, which excel in information gathering but fail to execute tasks. For manual execution, participants are allowed to seek assistance from online resources and people around them. For tasks carried out using Prompt2Task, participants can freely provide any information through textual prompts, which can either describe the task's goal or outline the detailed steps required for the task. After completing the task, participants are required to use a different expression to invoke the task again.

Metrics: We recorded the time and success rate for each task, both when performed manually and via Prompt2Task. For tasks carried out using Prompt2Task, the frequency of user interventions was also recorded.

Experimental Device: Participants used their own Android smartphones for the experiment. Prompt2Task's client is installed on these devices (detailed in Section 4.3).

In summary, we collected 192 ($12 \times 8 \times 2$) data points for tasks completed using Prompt2Task (half of which from first-time usage and the other half from second-time usage), and 96 (12×8) data points for tasks completed manually.

6.2 Experimental Results

This section demonstrates the experiment’s results, focusing on metrics including success rate, completion time, and frequency of user interventions, as well as subjective feedback.

Table 12. The task success rate (%) of different methods and participant groups. Tasks are considered successful only if completed within 10 minutes. “Prompt2Task (Second Time)” refers to tasks that have been previously executed and are being re-executed with a different textual prompt.

	Manual	Prompt2Task (First Time)	Prompt2Task (Second Time)
Unskilled	66.67	97.91	93.75
Skilled	77.08	97.91	97.91

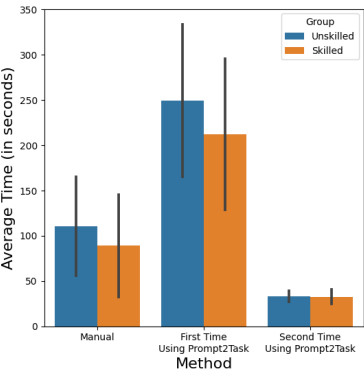


Fig. 14. Impact of methods and participant groups on task completion time (detailed in Table 14).

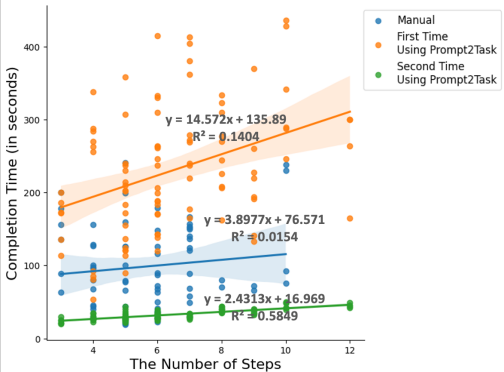


Fig. 15. Impact of step count on task completion time.

Success Rate. Table 12 reveals that Prompt2Task outperforms manual methods in both skilled and unskilled groups in terms of task success rate. Failures during the first use of Prompt2Task were largely attributable to the low quality of online tutorials for uncommonly used applications. Failures during the second attempt were mainly due to page data loading issues that blocked access to operation widgets.

Completion Time. Figure 14 shows that completing tasks with Prompt2Task takes significantly longer during the first use than when carried out manually (p -value<0.001). Figure 15 indicates that the completion time correlates positively with the number of steps involved during the first use of Prompt2Task, which is more pronounced than with manual methods and the second use of Prompt2Task. The time stems primarily from the system’s retrieval for online tutorials and exploration of operation sequences, which is tied to the efficiency limitations of the LLM. Notably, time decreases substantially on second use, as the system’s historical task repository eliminates the need for additional tutorial retrieval and exploration, thereby becoming significantly faster than manual methods (p -value<0.001).

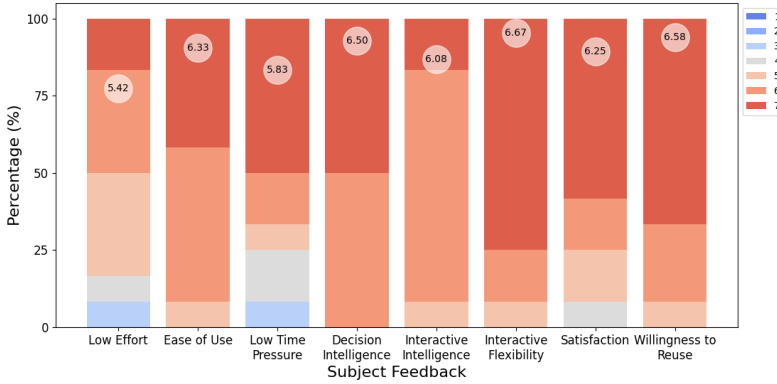


Fig. 16. Distribution of participant ratings for each statement in Table 15, assessing various aspects of Prompt2Task. Ratings range from 1 (strongly disagree) to 7 (strongly agree).

User Intervention. When using Prompt2Task, each new task required 0.92 user interventions (sd = 1.07) on average. Of these, 0.75 interventions per task (sd = 1.04) occurred during the **operation mapping**. Among the user intervention requests made by the *Assessment Agent*, 66.67% were deemed necessary. Notably, the second time a task was executed using Prompt2Task, user interventions were no longer required.

Subjective Feedback. Post-experiment interviews were conducted with participants, and their 7-point Likert scale responses are summarized in Figure 16. The participants’ ratings demonstrate that Prompt2Task was well-received across several dimensions, including low effort, ease of use, low time pressure, decision intelligence, interactive intelligence, interactive flexibility, satisfaction, and willingness to reuse. Additionally, the following insights were gathered from the interviews:

- **Preference for Forms of Intervention:** Participants had different experiences with various forms of intervention and expressed distinct expectations for improvement.

For the dialogue, most participants (11/12) were satisfied, finding it an effective way to express their intentions. P3 expressed surprise and satisfaction, “*Once I taught it the process, next time I only needed to provide a few keywords to complete the task.*” P1 found the context supplement very helpful, which allows for queries without the need for “*careful consideration*”. However, P1 also found the dialogues somewhat verbose, stating, “*Occasionally, the agent would ask me for unnecessary parameter details. For example, asking me for the WeChat version number—I don’t even know it. I would prefer the agent to generate some reliable options for me to choose from.*”

All participants indicated that they preferred demonstration over instruction editing. P2 commented, “*As long as the text isn’t too far off, I won’t modify it. Editing the text is less convenient than just demonstrating the operation later.*”

Regarding demonstrations, participants (10/12) commonly noted that most interventions were for locating small icons, which they accepted. For improvement, P1 suggested incorporating images into the step descriptions for clarity, similar to the online tutorials including screenshots. Some participants found that the agent’s prompts were sometimes incorrect, leaving them unsure about how to proceed. P8 mentioned, “*The agent asked me to click ‘workshop’, but there was nothing like that on the page.*” P4, an older user, remarked, “*Adding steps is quite challenging for me; I hope the tutorials could be more accurate.*”

In addition to the above interaction methods, P3 expressed a desire for more intuitive communication methods, *“Guess what I am trying to do and help me out when I am stuck in a certain step.”*

- **Tolerance for Latency:** Participants’ tolerance for latency varied depending on the individual, interaction state, and task. Most participants (10/12) acknowledged that some complex tasks understandably took longer. P5 noted, *“Although the process took a bit of time, the real-time feedback made it feel like a learning experience, which was acceptably paced for me.”* However, some participants expected lower latency for simpler tasks during their first use of Prompt2Task. P1 indicated that while he could accept the execution time, he hoped for faster response in chatting. P2 felt frustrated when the agent was slow in performing tasks she already knew how to complete. P4 remarked, *“It felt like I had to wait a bit for each step, which made me feel like I was just idling, even though it was only a few seconds. Perhaps some animations or interactive elements could help improve the experience.”* Fortunately, after using Prompt2Task a second time, all participants expressed satisfaction with the completion time.
- **Evaluation of Intelligence and Expectations:** All participants acknowledged the system’s intelligence. P3 specifically praised the feature that *“aligns the smartphone’s operational page with each step description”*. Similarly, P5 stated, *“Before, I struggled with the tutorial’s first step not matching my current page. Now it is a significant improvement.”* Room for improvement was also noted. P9 advocated for a more *“aggressive”* execution approach, arguing the current system is slightly conservative. P11 noted that some steps for confirmation were unnecessary. P12 observed, *“Tasks generally succeeded, but some took a roundabout path.”*
- **Application Scenarios:** Many participants praised the system’s transparency and user-centered approach. P8 stated, *“It gives me a feeling like I’m navigating through the tasks. It’s very interesting.”* The utility of Prompt2Task in simplifying otherwise complex tasks was another strong point. P1 expressed, *“I often ask my family how to use various functions on my phone, but it’s challenging to follow their instructions. Now I can just copy their responses and the system executes the steps for me.”* Similarly, P12 shared, *“I find new applications, especially those with complex functions, very daunting. It’s such a hassle to contact customer service and then follow their steps. Prompt2Task saves me from all that trouble.”* P9 pointed out, *“I can define a few routine tasks, like accounting after spending money. The agent can then automate these tasks for me.”* P3 stated, *“I often forget the steps, so the replay is very useful for me.”*

7 DISCUSSION

In this section, we delve deeper into the significance of knowledge and the applications of our proposed Prompt2Task. We also discuss the limitations of the system and potential future work.

7.1 Open and Accumulated Knowledge

Prompt2Task aims to convert information across modalities (from texts to operations), with its stages focusing on bridging gaps between them and minimizing information loss. In addition to the collaboration of its multi-agents, Prompt2Task’s effectiveness is derived from its open and accumulated knowledge, ensuring its broad applicability and efficiency.

Prompt2Task leverages massive online tutorials to address a broader range of user needs, surpassing existing work that relies on pre-collected datasets [49, 73, 75]. It supports functionalities deep within applications, which exceeds the capabilities of current assistants like Google Assistant and HUAWEI Celia. Moreover, Prompt2Task’s approach to mobile semantics is open-ended, enabling exploration beyond rigid adherence to given instructions. The open-ended approach allows Prompt2Task to handle diverse textual inputs and adapt to changing environments, significantly expanding the task automation’s application scope.

Table 13. “Outdated tutorials” is the result of using the 500 ground-truth step descriptions (100 step descriptions over 5 rounds of testing) as input. “Tutorials not found” is the evaluation of the entire pipeline, which used a total of 12,500 textual prompts (2,500 prompts over 5 rounds of testing) as input.

	Outdated Tutorials		Tutorials Not Found	
	Success Rate	User Intervention	Success Rate	User Intervention
Prompt2Task	75/155	-	611/1448	-
Prompt2Task(AK)	-	-	467/967	-
Prompt2Task(minimal UI+AK)	140/155	0.75(sd = 1.08)	698/855	0.68(sd = 1.22)
Prompt2Task(UI+AK)	140/155	0.75(sd = 1.08)	91/110	0.86(sd = 1.21)

Prompt2Task’s knowledge base evolves through user usage and feedback. The evolution manifests in improved success rates, rising from 60.80% to 95.24%, and reduced completion time for repeated tasks (Figure 14). Notably, the accumulation of knowledge contributes to a reduction in user interventions. When similar task requirements arise again, Prompt2Task operates autonomously, only requiring user intervention for contextual parameter adjustments or significant interface updates. The accumulated knowledge empowers Prompt2Task to tackle increasingly complex and nuanced tasks, dynamically adapting to emerging user needs.

It is natural to consider how Prompt2Task handles outdated knowledge. While accumulated knowledge enhances Prompt2Task’s performance by improving efficiency (Cases 1 and 4 in Appendix B.3), reducing user interaction in future tasks (Cases 2 and 3 in Appendix B.3), and increasing success rates (Case 5 in Appendix B.3), it is not indispensable. When a significant app update causes the accumulated knowledge outdated, Prompt2Task’s multi-agent design can still work effectively. As shown in Figures 5, 7, and 8, outdated or missing knowledge only affects specific parts. Therefore, the agents still have the potential to make correct decisions. Upon successful execution, the accumulated knowledge is updated.

As illustrated in Table 13, Prompt2Task demonstrates high success rates even with outdated tutorials or when no tutorials are found, indicating that the *Grounding Agent* does not entirely rely on tutorials. In the worst-case scenario, where no tutorials match, no useful accumulated knowledge is available, and no human intervention is provided, Prompt2Task still achieves a 42.20% success rate. Even if the system fails, stage-wise transparency allows users to help the agent complete the task correctly.

7.2 Applications

By incorporating massive external knowledge and accepting various textual inputs, Prompt2Task supports a wide range of task executions, benefiting both end users and developers.

For end users, Prompt2Task improves the usability of smartphones and facilitates assistance between peers. It offers guidance for users to navigate unfamiliar smartphone functions, enabling them to access and automate tasks deep within applications. It allows users to personalize their smartphone experience by automating their own complex or repetitive tasks. Furthermore, textual assistance from various sources, such as family, friends, or online customer service, can be fed to Prompt2Task for automating tasks, which is valuable for teamwork or external support. For instance, an elderly user receiving textual assistance from family members can simply input the text into Prompt2Task for automating tasks, without the need to repeatedly switch between text and complex interfaces.

For developers, Prompt2Task can reduce the costs of developing and maintaining task automation. It can cost-effectively transform a vast array of existing tutorial texts into operations, seamlessly

integrated into current voice assistants or online customer service, thereby expanding the range of supported tasks. Application and platform developers can leverage Prompt2Task to transform text-based cases in bulk into interactive tutorials or executable test scripts. It significantly reduces the cost of updating voice assistants, interactive tutorials, and test scripts, as developers need only update the source texts regularly to respond to the latest application functions and interfaces. Nonetheless, in sensitive applications such as banking, implementing robust authentication mechanisms is crucial to prevent malicious command execution.

Additionally, although Prompt2Task is designed for smartphones, its underlying methodology, particularly the multi-agent framework and extensive knowledge base, can be adapted to automate tasks in other domains, such as websites and IoT devices.

7.3 Limitations and Future Work

We outline the limitations of Prompt2Task, along with possible directions for future enhancements.

Adapting Dialogue Mechanisms for Diverse User Groups: The current design of the *Analysis Agent* offers a generalized approach without catering to the specific needs of different user demographics. For instance, older and younger users may express issues differently, necessitating a more nuanced, tailored dialogue system to accommodate their needs.

Parsing Complex Tutorial Logic: Automating responses to conditional branches like “If the version number is less than 14.0, please upgrade the app” could enhance the system’s efficiency. Future work will focus on more sophisticated logic analysis, including nested conditions.

Improving Semantic Understanding of GUI: In the present design, the mobile semantics module emphasizes the textual descriptions of mobile pages. However, current models often fall short, particularly when faced with pages not meeting accessibility standards. It results in blocking access to operation widgets and a notable loss of information during the translation from interface visuals to text. Moving forward, we aim to enhance algorithms that better recognize semantic structures within GUI, with the potential improvement of image recognition technologies.

Balancing Breadth with Depth: Currently, to expand the application scope, Prompt2Task applies the capabilities of LLM across various stages. However, some domains have not been deeply explored, such as addressing ambiguities in natural language (which the LLM’s rephrasing might not address) and setting instruction parsing templates to reduce the risk of LLM fabrication. As Prompt2Task’s usage data increases, we will systematically collect domain-specific data and user feedback to fine-tune the LLM or train local models, gradually reducing reliance on generalized templates and increasing its capacity for nuanced, domain-specific processing.

8 CONCLUSION

This paper presents Prompt2Task as an effective solution in UI task automation, particularly addressing the barriers of technical complexity, thereby facilitating its broader application. Utilizing the multi-agent framework and supplementing online tutorials, Prompt2Task is capable of interpreting unrestricted textual prompts to perform a wide range of automation tasks. As knowledge is accumulated from user usage and feedback, Prompt2Task improves task success rates and efficiency. Performance evaluations showcase a marked rise in success rates, ascending from an initial 22.28% to a remarkable 95.24%, with an average of only 0.69 user interventions per new task. Even without user interventions, a success rate of 78.16% is achieved. User studies further validate the effectiveness of Prompt2Task in delivering a user-friendly and intelligent experience. Upcoming efforts aim to minimize the burden of user interaction, engage a wider user group, and boost the system’s proficiency in managing complex tasks.

ACKNOWLEDGMENTS

This work is supported by the Natural Science Foundation of China under Grant No. 62132010, Beijing Key Lab of Networked Multimedia, Institute for Artificial Intelligence, Tsinghua University (THUI), Beijing National Research Center for Information Science and Technology (BNRist), 2025 Key Technological Innovation Program of Ningbo City under Grant No. 2022Z080, Beijing Municipal Science and Technology Commission, Administrative Commission of Zhongguancun Science Park No.Z221100006722018, and Science and Technology Innovation Key R&D Program of Chongqing.

REFERENCES

- [1] Santiago Aguirre and Alejandro Rodriguez. 2017. Automation of a Business Process Using Robotic Process Automation (RPA): A Case Study. In *Applied Computer Sciences in Engineering*, Juan Carlos Figueroa-García, Eduyn Ramiro López-Santana, José Luis Villa-Ramírez, and Roberto Ferro-Escobar (Eds.). Springer International Publishing, Cham, 65–71.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv:2204.01691 [cs.RO]*
- [3] Muna S. Al-Razgan, Hend S. Al-Khalifa, Mona D. Al-Shahrani, and Hessah H. AlAjmi. 2012. Touch-Based Mobile Phone Interface Guidelines and Design Recommendations for Elderly People: A Survey of the Literature. In *Neural Information Processing*, Tingwen Huang, Zhigang Zeng, Chuandong Li, and Chi Sing Leung (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 568–574.
- [4] Saleema Amershi, Maya Cakmak, William Bradley Knox, and Todd Kulesza. 2014. Power to the People: The Role of Humans in Interactive Machine Learning. *AI Magazine* 35, 4 (Dec. 2014), 105–120. <https://doi.org/10.1609/aimag.v35i4.2513> Number: 4.
- [5] Chongyang Bai, Xiaoxue Zang, Ying Xu, Srinivas Sunkara, Abhinav Rastogi, Jindong Chen, and Blaise Agüera y Arcas. 2021. UIbert: Learning Generic Multimodal Representations for UI Understanding. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, Zhi-Hua Zhou (Ed.). International Joint Conferences on Artificial Intelligence Organization, 1705–1712. <https://doi.org/10.24963/ijcai.2021/235>
- [6] Erin Beneteau, Olivia K. Richards, Mingrui Zhang, Julie A. Kientz, Jason Yip, and Alexis Hiniker. 2019. Communication Breakdowns Between Families and Alexa. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3290605.3300473>
- [7] Stuart Berg, Dominik Kutra, Thorben Kroeger, Christoph N Straehle, Bernhard X Kausler, Carsten Haubold, Martin Schiegg, Janez Ales, Thorsten Beier, Markus Rudy, et al. 2019. Ilastik: interactive machine learning for (bio) image analysis. *Nature methods* 16, 12 (2019), 1226–1232.
- [8] Tommaso Bolognesi and Ed Brinksma. 1987. Introduction to the ISO specification language LOTOS. *Computer Networks and ISDN Systems* 14, 1 (1987), 25–59. [https://doi.org/10.1016/0169-7552\(87\)90085-7](https://doi.org/10.1016/0169-7552(87)90085-7)
- [9] S.R.K. Branavan, Harr Chen, Luke Zettlemoyer, and Regina Barzilay. 2009. Reinforcement Learning for Mapping Instructions to Actions. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*. Association for Computational Linguistics, Suntec, Singapore, 82–90. <https://aclanthology.org/P09-1010>
- [10] Andrea Burns, Deniz Arsan, Sanjna Agrawal, Ranjitha Kumar, Kate Saenko, and Bryan A. Plummer. 2022. A Dataset for Interactive Vision-Language Navigation with Unknown Command Feasibility. <http://arxiv.org/abs/2202.02312> *arXiv:2202.02312 [cs]*.
- [11] Michelle Carney, Barron Webster, Irene Alvarado, Kyle Phillips, Noura Howell, Jordan Griffith, Jonas Jongejan, Amit Pitaru, and Alexander Chen. 2020. Teachable Machine: Approachable Web-Based Tool for Exploring Machine Learning Classification. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI EA '20). Association for Computing Machinery, New York, NY, USA, 1–8. <https://doi.org/10.1145/3334480.3382839>
- [12] Jun Chen, Han Guo, Kai Yi, Boyang Li, and Mohamed Elhoseiny. 2022. VisualGPT: Data-efficient Adaptation of Pretrained Language Models for Image Captioning. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 18009–18019. <https://doi.org/10.1109/CVPR52688.2022.01750>

- [13] Jieshan Chen, Amanda Swearngin, Jason Wu, Titus Barik, Jeffrey Nichols, and Xiaoyi Zhang. 2022. Towards Complete Icon Labeling in Mobile Applications. In *CHI Conference on Human Factors in Computing Systems*. ACM, New Orleans LA USA, 1–14. <https://doi.org/10.1145/3491102.3502073>
- [14] Xiaou Chen, Fei Wang, Zhenwei You, Xiaochun Wang, Chunjing Tao, and Jian Liu. 2017. Design of Interactive Tutorials on Mobile Applications for Chinese Middle-Aged and Older Adults. *Art and Design Review* 05, 03 (2017), 162. <https://doi.org/10.4236/adr.2017.53013> Number: 03 Publisher: Scientific Research Publishing.
- [15] Yi Chen, Rui Wang, Haiyun Jiang, Shuming Shi, and Ruifeng Xu. 2023. Exploring the Use of Large Language Models for Reference-Free Text Quality Evaluation: An Empirical Study. [arXiv:2304.00723](https://arxiv.org/abs/2304.00723) [cs.CL]
- [16] André Dantas de Medeiros, Nayara Pereira Capobianco, Jose Maria da Silva, Laercio Junio da Silva, Clissia Barboza da Silva, and Denise Cunha Fernandes dos Santos Dias. 2020. Interactive machine learning for soybean seed and seedling quality classification. *Scientific reports* 10, 1 (2020), 11267.
- [17] Biplob Deka, Zifeng Huang, Chad Franzen, Joshua Hibschan, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A Mobile App Dataset for Building Data-Driven Design Applications. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*. ACM, Québec City QC Canada, 845–854. <https://doi.org/10.1145/3126594.3126651>
- [18] Changyu Deng, Xunbi Ji, Colton Rainey, Jianyu Zhang, and Wei Lu. 2020. Integrating Machine Learning with Human Knowledge. *iScience* 23, 11 (Nov. 2020), 101656. <https://doi.org/10.1016/j.isci.2020.101656>
- [19] Morgan Dixon, Gierad Laput, and James Fogarty. 2014. Pixel-based methods for widget state and style in a runtime implementation of sliding widgets. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) (*CHI '14*). Association for Computing Machinery, New York, NY, USA, 2231–2240. <https://doi.org/10.1145/2556288.2556979>
- [20] Ali Dorri, Salil S. Kanhere, and Raja Jurdak. 2018. Multi-Agent Systems: A Survey. *IEEE Access* 6 (2018), 28573–28593. <https://doi.org/10.1109/ACCESS.2018.2831228>
- [21] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. [arXiv:2305.14325](https://arxiv.org/abs/2305.14325) [cs.CL]
- [22] John J. Dudley and Per Ola Kristensson. 2018. A Review of User Interface Design for Interactive Machine Learning. *ACM Trans. Interact. Intell. Syst.* 8, 2, Article 8 (jun 2018), 37 pages. <https://doi.org/10.1145/3185517>
- [23] Jerry Alan Fails and Dan R. Olsen. 2003. Interactive Machine Learning. In *Proceedings of the 8th International Conference on Intelligent User Interfaces* (Miami, Florida, USA) (*IUI '03*). Association for Computing Machinery, New York, NY, USA, 39–45. <https://doi.org/10.1145/604045.604056>
- [24] Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. 2022. Language-agnostic BERT Sentence Embedding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 878–891. <https://doi.org/10.18653/v1/2022.acl-long.62>
- [25] Sidong Feng and Chunyang Chen. 2024. Prompting Is All You Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (*ICSE '24*). Association for Computing Machinery, New York, NY, USA, Article 67, 13 pages. <https://doi.org/10.1145/3597503.3608137>
- [26] C. Ailie Fraser, Tricia J. Ngoon, Mira Dontcheva, and Scott Klemmer. 2019. RePlay: Contextually Presenting Learning Videos Across Software Applications. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (*CHI '19*). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3290605.3300527>
- [27] Jonathan Grudin and Richard Jacques. 2019. Chatbots, Humbots, and the Quest for Artificial General Intelligence. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (*CHI '19*). Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3290605.3300439>
- [28] Zecheng He, Srinivas Sunkara, Xiaoxue Zang, Ying Xu, Lijuan Liu, Nevan Wichers, Gabriel Schubiner, Ruby Lee, and Jindong Chen. 2021. ActionBert: Leveraging User Actions for Semantic Understanding of User Interfaces. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 7 (May 2021), 5931–5938. <https://doi.org/10.1609/aaai.v35i7.16741> Number: 7.
- [29] Lukas-Valentin Herm, Christian Janiesch, Alexander Helm, Florian Imgrund, Adrian Hofmann, and Axel Winkelmann. 2023. A framework for implementing robotic process automation projects. *Information Systems and e-Business Management* 21, 1 (2023), 1–35.
- [30] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, and Chenglin Wu. 2023. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. [arXiv:2308.00352](https://arxiv.org/abs/2308.00352) [cs.AI]
- [31] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. 2023. CogAgent: A Visual Language Model for GUI

- Agents. arXiv:2312.08914 [cs.CV]
- [32] Forrest Huang, Gang Li, Tao Li, and Yang Li. 2024. Automatic Macro Mining from Interaction Traces at Scale. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 1038, 16 pages. <https://doi.org/10.1145/3613904.3642074>
 - [33] Tian Huang, Chun Yu, Weinan Shi, Bowen Wang, David Yang, Yihao Zhu, Zhaoheng Li, and Yuanchun Shi. 2023. Interaction Proxy Manager: Semantic Model Generation and Run-Time Support for Reconstructing Ubiquitous User Interfaces of Mobile Services. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 7, 3, Article 99 (sep 2023), 39 pages. <https://doi.org/10.1145/3610929>
 - [34] Rashedul Islam, Rofiqul Islam, and Tohidul Mazumder. 2010. Mobile application and its global impact. *International Journal of Engineering & Technology* 10, 6 (2010), 72–78.
 - [35] Bernard J. Jansen, Amanda Spink, Judy Bateman, and Tefko Saracevic. 1998. Real life information retrieval: a study of user queries on the Web. *ACM SIGIR Forum* 32, 1 (April 1998), 5–17. <https://doi.org/10.1145/281250.281253>
 - [36] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *ACM Comput. Surv.* 55, 12, Article 248 (mar 2023), 38 pages. <https://doi.org/10.1145/3571730>
 - [37] Xiaofu Jin, Xiaozhu Hu, Xiaoying Wei, and Mingming Fan. 2022. Synapse: Interactive Guidance by Demonstration with Trial-and-Error Support for Older Adults to Use Smartphone Apps. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6, 3, Article 121 (sep 2022), 24 pages. <https://doi.org/10.1145/3550321>
 - [38] Stefan Z Jovanović, Jelena S Đurić, and Tatjana V Šibalića. 2018. Robotic process automation: overview and opportunities. *International Journal Advanced Quality* 46, 3-4 (2018), 34–39.
 - [39] Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, Julian Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2022. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. <http://arxiv.org/abs/2210.03347> arXiv:2210.03347 [cs].
 - [40] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2023. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 18893–18912. <https://proceedings.mlr.press/v202/lee23g.html>
 - [41] Gang Li and Yang Li. 2023. Spotlight: Mobile UI Understanding using Vision-Language Models with a Focus. arXiv:2209.14927
 - [42] Guo Li, Tun Lu, Jiang Yang, Xiaomu Zhou, Xianghua Ding, and Ning Gu. 2015. Intelligently Creating Contextual Tutorials for GUI Applications. In *2015 IEEE 12th Intl Conf on Ubiquitous Intelligence and Computing and 2015 IEEE 12th Intl Conf on Autonomic and Trusted Computing and 2015 IEEE 15th Intl Conf on Scalable Computing and Communications and Its Associated Workshops (UIC-ATC-ScalCom)*. IEEE, Beijing, 187–196. <https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCCom-IoP.2015.50>
 - [43] Tao Li, Gang Li, Jingjie Zheng, Purple Wang, and Yang Li. 2024. MUG: Interactive Multimodal Grounding on User Interfaces. In *Findings of the Association for Computational Linguistics: EACL 2024*, Yvette Graham and Matthew Purver (Eds.). Association for Computational Linguistics, St. Julian's, Malta, 231–251. <https://aclanthology.org/2024.findings-eacl.17>
 - [44] Toby Jia-Jun Li, Amos Azaria, and Brad A. Myers. 2017. SUGILITE: Creating Multimodal Smartphone Automation by Demonstration. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 6038–6049. <https://doi.org/10.1145/3025453.3025483>
 - [45] Toby Jia-Jun Li, Igor Labutov, Xiaohan Nancy Li, Xiaoyi Zhang, Wenze Shi, Wanling Ding, Tom M. Mitchell, and Brad A. Myers. 2018. APPINITE: A Multi-Modal Interface for Specifying Data Descriptions in Programming by Demonstration Using Natural Language Instructions. In *2018 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 105–114. <https://doi.org/10.1109/VLHCC.2018.8506506> ISSN: 1943-6106.
 - [46] Toby Jia-Jun Li, Lindsay Popowski, Tom Mitchell, and Brad A Myers. 2021. Screen2Vec: Semantic Embedding of GUI Screens and GUI Components. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–15. <https://doi.org/10.1145/3411764.3445049>
 - [47] Toby Jia-Jun Li and Oriana Riva. 2018. Kite: Building Conversational Bots from Mobile Apps. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services* (Munich, Germany) (MobiSys '18). Association for Computing Machinery, New York, NY, USA, 96–109. <https://doi.org/10.1145/3210240.3210339>
 - [48] Wei Li. 2021. Learning UI Navigation through Demonstrations composed of Macro Actions. <http://arxiv.org/abs/2110.08653> arXiv:2110.08653 [cs].

- [49] Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge. 2020. Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 8198–8210. <https://doi.org/10.18653/v1/2020.acl-main.729>
- [50] Yang Li, Gang Li, Luheng He, Jingjie Zheng, Hong Li, and Zhiwei Guan. 2020. Widget Captioning: Generating Natural Language Description for Mobile User Interface Elements. <http://arxiv.org/abs/2010.04295> arXiv:2010.04295 [cs].
- [51] Zhangheng Li, Keen You, Haotian Zhang, Di Feng, Harsh Agrawal, Xiujuan Li, Mohana Prasad Sathya Moorthy, Jeff Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-UI 2: Mastering Universal User Interface Understanding Across Platforms. arXiv:2410.18967 [cs.CV] <https://arxiv.org/abs/2410.18967>
- [52] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujia Yang, Zhaopeng Tu, and Shuming Shi. 2023. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. arXiv:2305.19118 [cs.CL]
- [53] Quentin Limbourg and Jean Vanderdonckt. 2004. Comparing task models for user interface design. *The handbook of task analysis for human-computer interaction* 6 (2004), 135–154.
- [54] Thomas F. Liu, Mark Craft, Jason Situ, Ersin Yumer, Radomir Mech, and Ranjitha Kumar. 2018. Learning Design Semantics for Mobile Apps. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology* (Berlin, Germany) (*UIST '18*). Association for Computing Machinery, New York, NY, USA, 569–579. <https://doi.org/10.1145/3242587.3242650>
- [55] Pin Ni, Yuming Li, Gangmin Li, and Victor Chang. 2020. Natural language understanding approaches based on joint task of intent detection and slot filling for IoT voice interaction. *Neural Comput. Appl.* 32, 20 (Oct. 2020), 16149–16166. <https://doi.org/10.1007/s00521-020-04805-x>
- [56] Norman Di Palo, Arunkumar Byravan, Leonard Hasenclever, Markus Wulfmeier, Nicolas Heess, and Martin Riedmiller. 2023. Towards A Unified Agent with Foundation Models. arXiv:2307.09668 [cs.RO]
- [57] Lihang Pan, Chun Yu, JiaHui Li, Tian Huang, Xiaojun Bi, and Yuanchun Shi. 2022. Automatically Generating and Improving Voice Command Interface from Operation Sequences on Smartphones. In *CHI Conference on Human Factors in Computing Systems*. ACM, New Orleans LA USA, 1–21. <https://doi.org/10.1145/3491102.3517459>
- [58] Liviu Panait and Sean Luke. 2005. Cooperative multi-agent learning: The state of the art. *Autonomous agents and multi-agent systems* 11 (2005), 387–434. <https://doi.org/10.1007/s10458-005-2631-2>
- [59] Stefano Passini, Filiberto Strazzari, and Annamaria Borghi. 2008. Icon-function relationship in toolbar icons. *Displays* 29, 5 (2008), 521–525. <https://doi.org/10.1016/j.displa.2008.07.001>
- [60] Panupong Pasupat, Tian-Shun Jiang, Evan Zheran Liu, Kelvin Guu, and Percy Liang. 2018. Mapping Natural Language Commands to Web Elements. arXiv:1808.09132 [cs.CL]
- [61] Fabio Paternò, Cristiano Mancini, and Silvia Meniconi. 1997. ConcurTaskTrees: A Diagrammatic Notation for Specifying Task Models. In *Proceedings of the IFIP TC13 International Conference on Human-Computer Interaction (INTERACT '97)*. Chapman & Hall, Ltd., GBR, 362–369. https://doi.org/10.1007/978-0-387-35175-9_58
- [62] Suporn Pongnumkul, Mira Dontcheva, Wilnot Li, Jue Wang, Lubomir Bourdev, Shai Avidan, and Michael F. Cohen. 2011. Pause-and-Play: Automatically Linking Screencast Video Tutorials with Applications. In *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology* (Santa Barbara, California, USA) (*UIST '11*). Association for Computing Machinery, New York, NY, USA, 135–144. <https://doi.org/10.1145/2047196.2047213>
- [63] Jorge Ribeiro, Rui Lima, Tiago Eckhardt, and Sara Paiva. 2021. Robotic Process Automation and Artificial Intelligence in Industry 4.0 – A Literature review. *Procedia Computer Science* 181 (2021), 51–58. <https://doi.org/10.1016/j.procs.2021.01.104>
- [64] August-Wilhelm Scheer, Oliver Thomas, and Otmar Adam. 2005. Process Modeling using Event-Driven Process Chains. *Process-Aware Information Systems: Bridging People and Software through Process Technology* (10 2005), 119 – 145. <https://doi.org/10.1002/0471741442.ch6>
- [65] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL]
- [66] Yiqiu Shen, Laura Heacock, Jonathan Elias, Keith Hentel, Beatriu Reig, George Shih, and Linda Moy. 2023. ChatGPT and Other Large Language Models Are Double-edged Swords. *Radiology* 307 (01 2023), e230163. <https://doi.org/10.1148/radiol.230163>
- [67] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 11523–11530. <https://doi.org/10.1109/ICRA48891.2023.10161317>
- [68] Yunpeng Song, Yiheng Bian, Yongtao Tang, Guiyu Ma, and Zhongmin Cai. 2024. VisionTasker: Mobile Task Automation Using Vision Based UI Understanding and LLM Task Planning. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (*UIST '24*). Association for Computing Machinery, New York, NY, USA, Article 49, 17 pages. <https://doi.org/10.1145/3654777.3676386>

- [69] Fabian Sperrle, Rita Sevastjanova, Rebecca Kehlbeck, and Mennatallah El-Assady. 2019. VIANA: Visual Interactive Annotation of Argumentation. In *2019 IEEE Conference on Visual Analytics Science and Technology (VAST)*. 11–22. <https://doi.org/10.1109/VAST47406.2019.8986917>
- [70] Neville A. Stanton. 2006. Hierarchical task analysis: Developments, applications, and extensions. *Applied Ergonomics* 37, 1 (2006), 55–79. <https://doi.org/10.1016/j.apergo.2005.06.003>
- [71] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. 2024. AXNav: Replaying Accessibility Tests from Natural Language. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI '24*). Association for Computing Machinery, New York, NY, USA, Article 962, 16 pages. <https://doi.org/10.1145/3613904.3642777>
- [72] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents. arXiv:2306.03314 [cs.AI]
- [73] Sagar Gubbi Venkatesh, Partha Talukdar, and Srinu Narayanan. 2022. UGIF: UI Grounded Instruction Following. <http://arxiv.org/abs/2211.07615> arXiv:2211.07615 [cs].
- [74] Minh Duc Vu, Han Wang, Jieshan Chen, Zhuang Li, Shengdong Zhao, Zhenchang Xing, and Chunyang Chen. 2024. GPTVoiceTasker: Advancing Multi-step Mobile Task Efficiency Through Dynamic Interface Exploration and Learning. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM, 1–17. <https://doi.org/10.1145/3654777.3676356>
- [75] Minh Duc Vu, Han Wang, Zhuang Li, Gholamreza Haffari, Zhenchang Xing, and Chunyang Chen. 2023. Voicify Your UI: Towards Android App Control with Voice Commands. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 7, 1, Article 44 (mar 2023), 22 pages. <https://doi.org/10.1145/3581998>
- [76] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling Conversational Interaction with Mobile UI using Large Language Models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (*CHI '23*). Association for Computing Machinery, New York, NY, USA, Article 432, 17 pages. <https://doi.org/10.1145/3544548.3580895>
- [77] Bryan Wang, Gang Li, Xin Zhou, Zhouong Chen, Tovi Grossman, and Yang Li. 2021. Screen2Words: Automatic Mobile UI Summarization with Multimodal Learning. In *The 34th Annual ACM Symposium on User Interface Software and Technology*. ACM, Virtual Event USA, 498–510. <https://doi.org/10.1145/3472749.3474765>
- [78] Cheng-Yao Wang, Wei-Chen Chu, Hou-Ren Chen, Chun-Yen Hsu, and Mike Y. Chen. 2014. EverTutor: automatically creating interactive guided tutorials on smartphones by user demonstration. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Toronto Ontario Canada, 4027–4036. <https://doi.org/10.1145/2556288.2557407>
- [79] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. arXiv:2401.16158 [cs.CL] <https://arxiv.org/abs/2401.16158>
- [80] Jimmy Wei, Kurt Shuster, Arthur Szlam, Jason Weston, Jack Urbanek, and Mojtaba Komeili. 2023. Multi-Party Chat: Conversational Agents in Group Settings with Humans and Models. arXiv:2304.13835 [cs.CL]
- [81] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered Task Automation in Android. arXiv:2308.15272
- [82] Ji-Rong Wen, Jian-Yun Nie, and Hong-Jiang Zhang. 2001. Clustering User Queries of a Search Engine. In *Proceedings of the 10th International Conference on World Wide Web* (Hong Kong, Hong Kong) (*WWW '01*). Association for Computing Machinery, New York, NY, USA, 162–168. <https://doi.org/10.1145/371920.371974>
- [83] Judith Wewerka, Sebastian Dax, and Manfred Reichert. 2020. A User Acceptance Model for Robotic Process Automation. In *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*. 97–106. <https://doi.org/10.1109/EDOC49727.2020.00021>
- [84] Thomas D. White, Gordon Fraser, and Guy J. Brown. 2019. Improving random GUI testing with image-based widget detection. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Beijing, China) (*ISSTA 2019*). Association for Computing Machinery, New York, NY, USA, 307–317. <https://doi.org/10.1145/3293882.3330551>
- [85] Jason Wu, Rebecca Krosnick, Eldon Schoop, Amanda Swearngin, Jeffrey P Bigham, and Jeffrey Nichols. 2023. Never-Ending Learning of User Interfaces. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (*UIST '23*). Association for Computing Machinery, New York, NY, USA, Article 113, 13 pages. <https://doi.org/10.1145/3586183.3606824>
- [86] Mulong Xie, Sidong Feng, Zhenchang Xing, Jieshan Chen, and Chunyang Chen. 2020. UIED: a hybrid tool for GUI element detection. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) (*ESEC/FSE 2020*). Association for Computing Machinery, New York, NY, USA, 1655–1659. <https://doi.org/10.1145/3368089.3417940>

[87] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629* [cs.CL]

[88] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LXIV* (Milan, Italy). Springer-Verlag, Berlin, Heidelberg, 240–255. https://doi.org/10.1007/978-3-031-73039-9_14

[89] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B. Tenenbaum, Tianmin Shu, and Chuang Gan. 2023. Building Cooperative Embodied Agents Modularly with Large Language Models. *arXiv:2307.02485* [cs.AI]

[90] Zhizheng Zhang, Xiaoyi Zhang, Wenxuan Xie, and Yan Lu. 2023. Responsible Task Automation: Empowering Large Language Models as Responsible Task Automators. *arXiv:2306.01242* [cs.AI]

[91] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2023. ExpeL: LLM Agents Are Experiential Learners. *arXiv:2308.10144* [cs.LG]

[92] Jason Chen Zhao, Richard C. Davis, Pin Sym Foong, and Shengdong Zhao. 2015. CoFaqade: A Customizable Assistive Approach for Elders and Their Helpers. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) (*CHI '15*). Association for Computing Machinery, New York, NY, USA, 1583–1592. <https://doi.org/10.1145/2702123.2702588>

[93] Mingyuan Zhong, Gang Li, Peggy Chi, and Yang Li. 2021. HelpViz: Automatic Generation of Contextual Visual Mobile Tutorials from Text-Based Instructions. In *The 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21)*. Association for Computing Machinery, New York, NY, USA, 1144–1153. <https://doi.org/10.1145/3472749.3474812>

A MULT-AGENT DESIGN

The following provides the technical and prompt details for each agent.

A.1 Analysis Agent

The agent processes the input through GPT-4 to produce a function description and step description as reflected in the input, with prompt details shown in Figure 17.

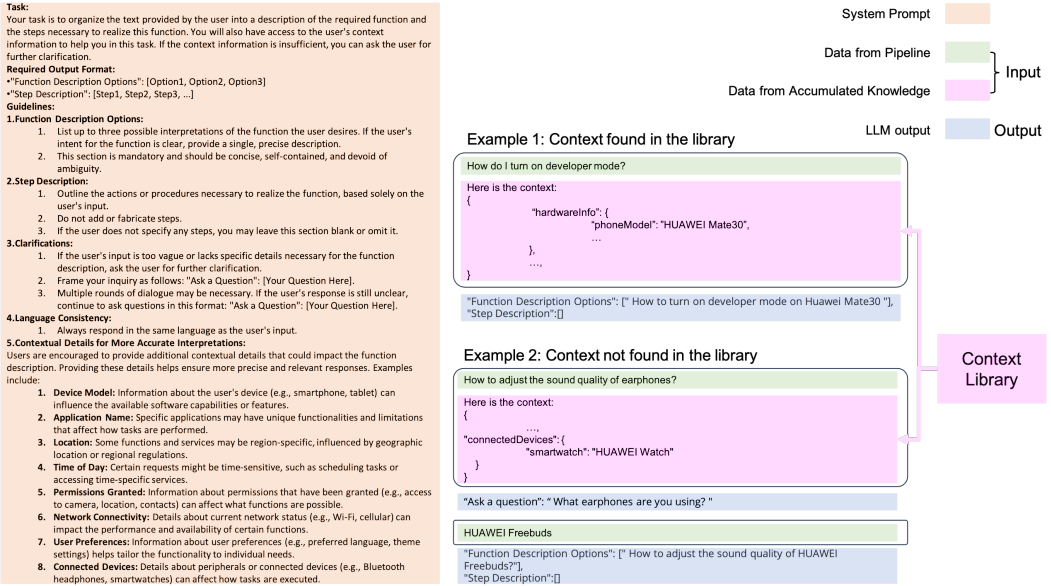


Fig. 17. Prompts used by the Analysis Agent.

A.2 Retrieval Agent

The agent uses GPT-4 to extract the step description from the raw text of search results, with prompt details shown in Figure 18.

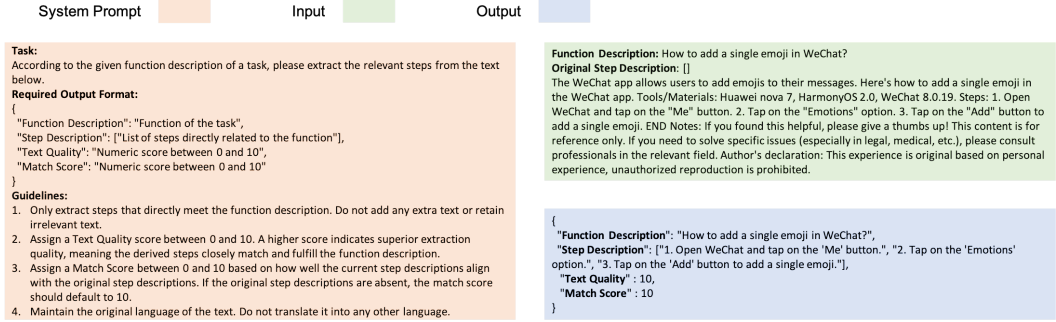


Fig. 18. Prompts used by the *Retrieval Agent*.

A.3 Parsing Agent

The agent uses GPT-4 to accomplish the transformation from step descriptions in natural language to formalized instructions, with prompt details shown in Figure 19.

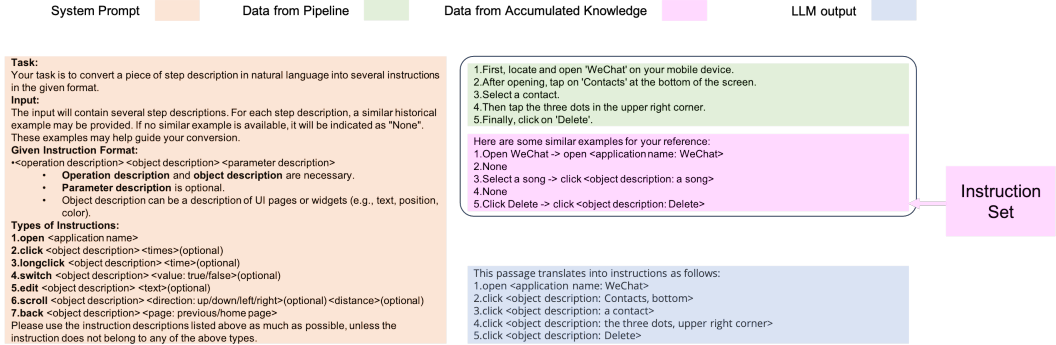


Fig. 19. Prompts used by the *Parsing Agent*.

A.4 Grounding Agent

In this section, we provide detailed descriptions of the strategies employed by the *Grounding Agent*. Initially, based on the current page's layout hierarchy, the agent identifies and selects all widgets that either possess text descriptions or have interactive attributes, thereby constructing a set of potential operation candidates. Subsequently, the agent employs the following strategies to predict the operational widget within these candidates:

- (1) **Historical task invocation:** If the instruction has been previously mapped to an operation, the agent can refer to stored widget data in the historical task repository. It enables the agent to direct the operation to a corresponding widget on the current interface. To evaluate

the similarity between the candidate widget and the stored widget, we employ a heuristic rule-based similarity function, as described in the work of Huang et al. [33]. This function considers 26 features of a widget, including its style and position in images, its style and position in layouts, and its role in page segmentation. The similarity thresholds have been established using Receiver Operating Characteristic (ROC) curve analysis.

- (2) **New instruction-based grounding:** For new instructions, the agent then employs the following strategies:
 - (a) **Strict text matching:** It involves searching for a widget whose text closely matches the object description (Instruction.Object). Similarity is defined as Equation 2, where Widget.text denotes the text and content description contained in the widget's layout information, and LCS signifies the length of the longest common sub-string between two strings. A match is confirmed only if the similarity exceeds an empirically preset threshold.

$$Sim(Widget, Instruction.Object) = \frac{2 \times LCS(Widget.text, Instruction.Object.text)}{length(Widget.text) + length(Instruction.Object.text)} \quad (2)$$

- (b) **Redirect, Add, Skip, Expand, Block:** The agent enumerates potential widgets with relevant information, including their text, content description, position on the current page, interactive attributes and so on. Subsequently, the LLM considers the current context to choose a strategy and then predicts the most suitable widget for executing the operation. Details of the prompt are illustrated in Figure 20.

A.5 Assessment Agent

The *Assessment Agent* utilizes the LLM with a rich set of contextual information to make decisions: follow, change, retract, or finish, while also describing if it is confident for its decision, as shown in Figure 21. If the agent is unconfident in its decision, the system automatically prompts the user for assistance.

B ACCUMULATED KNOWLEDGE

B.1 Construction of the Historical Task Repository

The Historical Task Repository serves as an archive for successfully executed automation tasks. Since the *Grounding Agent* doesn't strictly adhere to pre-generated instructions, the repository focuses on storing the actual sequences of operations executed. This subsection details how these operation are reverse-engineered into corresponding instructions and step descriptions.

- (1) **Operation Sequence Storage:** The first step is to archive the actual sequence of operations executed by the *Grounding Agent*, which is the most reliable record of the automation task. The system archives all layout hierarchical information and screenshots of the pages visited. It also logs the operation object's index in the page's layout tree, which can be used to retrieve the object's layout information. Additionally, since the object's position is recorded, it allows for capturing its corresponding image on the full-page screenshot.
- (2) **Reverse Engineering to Instructions:** Using the stored operation sequence, the system reverse-engineers to produce the corresponding instructions. Each operation is mapped to the potential instruction that could have prompted it, adhering to the format specified in Table 2. In this context, the operation types and parameters have already been logged by the system during the triggering of the operation. The descriptions of operation objects are derived from their texts, content descriptions, positions, and the average RGB values of their corresponding images.

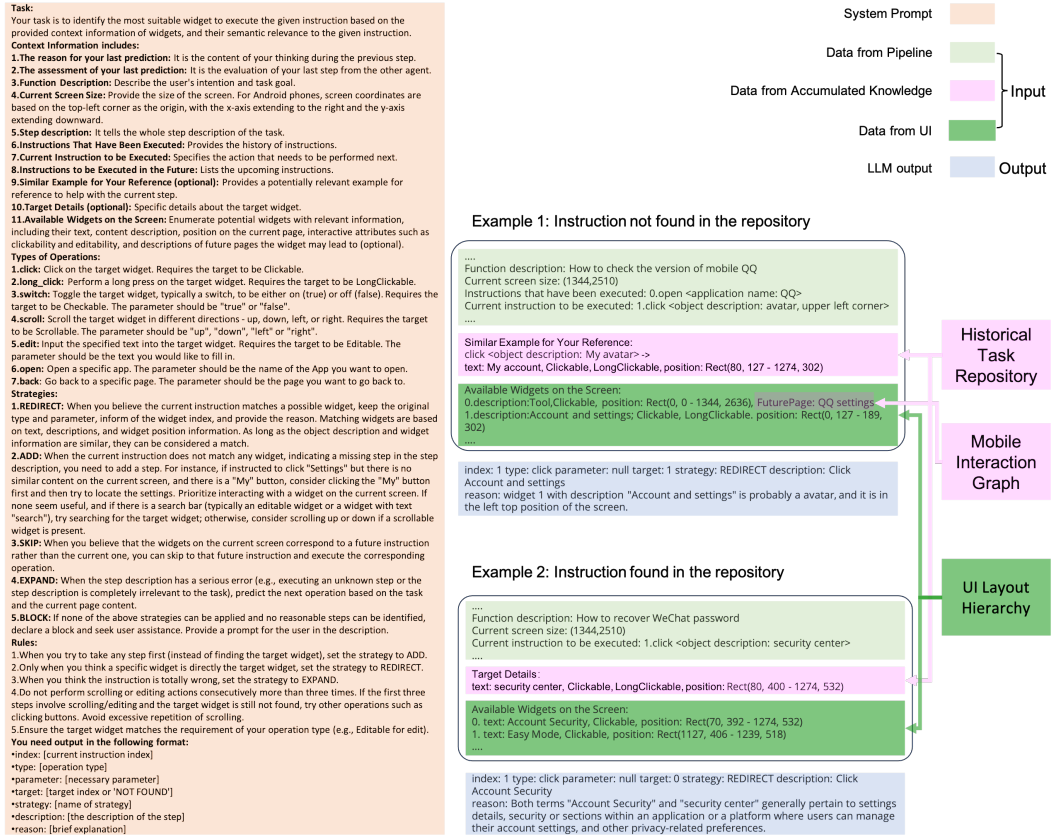


Fig. 20. Prompts used by the *Grounding Agent*.

- (3) **Generation of Step Descriptions:** Alongside instructions, the system also formulates step descriptions. They are rephrased versions of the instructions, articulated in natural language, generated by GPT-4.

B.2 Construction of the Context Library

The context library plays a crucial role in enhancing the system's capacity to interpret vague textual prompts by providing an array of mobile-specific parameters. Here is the method employed for constructing the library.

- (1) **Online Q&A Data Collection:** Initially, we sourced a set of 20,265 questions related to mobile phone usage from a recognized online Q&A platform⁷.
- (2) **Participant Involvement:** The collected questions were distributed among 24 undergraduate college students who participated in a formative study.
- (3) **Contextual Parameter Extraction:** Each participant was tasked with identifying the "central" keywords in each question. They then examined various variations of the question to infer the contextual parameters required for comprehensive understanding.

⁷<https://zhidao.baidu.com/>

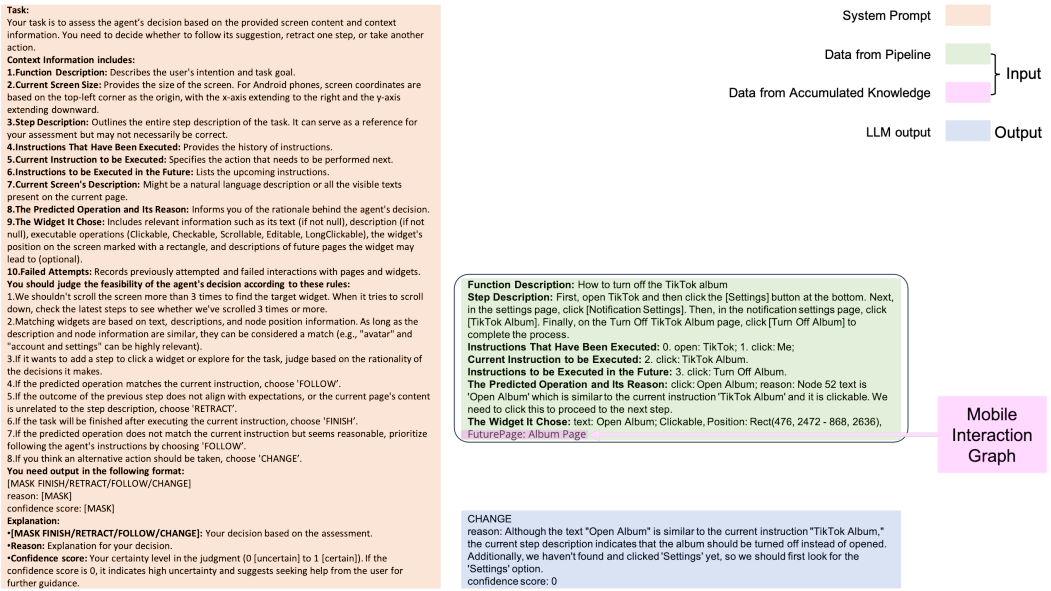


Fig. 21. Prompts used by the *Assessment Agent*.

(4) **Context Library Formation:** We employed experts to review, analyze, and categorize the list of contextual parameters identified by the participants. Their insights were integrated to develop the finalized context library.

Here is an example of records in the context library. Each time a task is successfully completed, the context library integrates new information using GPT-4, with prompt detailed in Figure 22.

```
{
  "hardwareInfo": {
    "deviceName": "mySmartPhone",
    "phoneModel": "HUAWEI Mate30",
    "battery": "0.23",
    "availableWifi": ["Network1", "Network2"]
  },
  "systemSetting": {
    "display": {
      "darkmode": true,
      "autoBrightness": true
    }
  },
  "privacyAndSecurity": {
    "locationAllowedServices": {
      "allowed": ["Maps", "WeChat", "Lark"]
    },
    "permissions": {
      "camera": true,
      "location": true,

```

```
        "contacts": false
    },
    "usingApp": ["App1", "App2"],
    "operatingSystem": "HarmonyOS 2.0",
    "location": "Beijing, China",
    "timeOfDay": "2023-04-12T14:30:00Z",
    "networkConnectivity": "Wi-Fi",
    "userPreferences": {
        "preferredLanguage": "en",
        "themeSettings": "dark"
    },
    "connectedDevices": {
        "bluetoothHeadphones": "HUAWEI Freebuds",
        "smartwatch": "HUAWEI Watch"
    }
}
```

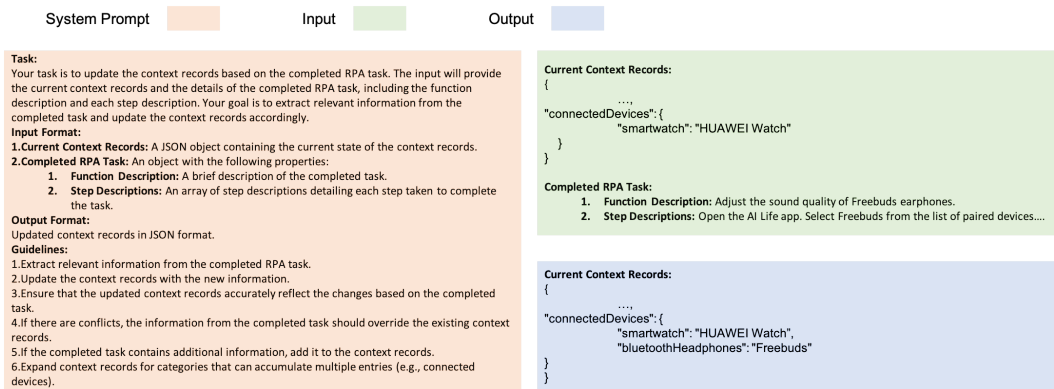


Fig. 22. Prompts used to update the context library based on the currently completed automation task.

B.3 Typical Cases

This section illustrates typical cases where accumulated knowledge optimizes the system in various ways, while also explaining how the system handles outdated knowledge.

B.3.1 Case 1. Case 1 demonstrates the optimization of the *Grounding Agent* by the historical task repository.

Suppose the current instruction is <type: click, object: “Account”>. As shown in Figure 23, the current UI suggests that “Me” should be clicked based on semantic similarity. During the initial execution, the *Grounding Agent* uses the LLM and chooses the redirect strategy to locate the target widget. This process is relatively slow and may result in inconsistent outcomes. If the target widget cannot be found, the *Grounding Agent* will choose to block, prompting the user to demonstrate the correct operation.

After the first execution, in subsequent executions, the system leverages the Prompt2Task model stored in the historical task repository. Knowing that this step is clicking “Me”, the *Grounding Agent*

can directly locate the target widget using heuristic rules, resulting in faster and more reliable outcomes.

If the app undergoes significant updates such that “Me” no longer exists or the widget style changes drastically, making it difficult to locate the widget using heuristic rules, the *Grounding Agent* will refer to the instruction <type: click, object: “Me”> and use the LLM to infer the operation again (using strategies such as redirect, add, skip, expand, or block). Even if the knowledge in the historical task repository is outdated, it still provides guidance by including more target details in the prompt, as shown in the prompt engineering of the *Grounding Agent* in Appendix A.4. After completing the new task, the historical task repository will store the updated target details.

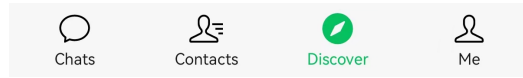


Fig. 23. Case 1: The instruction indicates clicking “Account”, but the current UI suggests that “Me” should be clicked instead.

B.3.2 Case 2. Case 2 demonstrates the optimization of the mobile semantics module in the *Grounding Agent* by the historical task repository, which then enhances the agent’s prediction.

Suppose the current instruction is <type: click, object: “more”>, and the current UI is as shown in Figure 24. During the initial execution, the agent is unable to identify “more” due to insufficient UI information (no text labels and pixel-based recognition failed). Therefore, the *Grounding Agent* enters a block state and requests the user to demonstrate the correct operation.

After the first execution, in subsequent executions, the mobile semantics module learns from the Prompt2Task model stored in the historical task repository and knows that the icon image corresponds to the “more” label. Therefore, it can provide this information to the *Grounding Agent*, which can then use strict text matching to directly locate and click the widget.

As long as the icon image does not change significantly, the mobile semantics module can label the icon as “more”. Otherwise, the process of requiring user intervention to identify the icon will be repeated, and after task completion, the historical task repository will be updated accordingly.



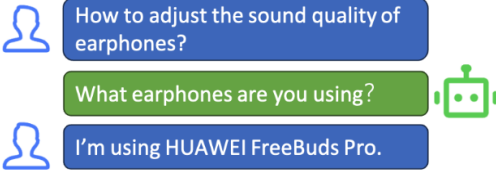
Fig. 24. Case 2: The instruction indicates clicking “more”, but the icon’s information is insufficient for the agent to locate it, likely requiring user demonstration upon first encounter.

B.3.3 Case 3. Case 3 demonstrates the optimization of the *Analysis Agent* by the context library.

As shown in Figure 25, the user input is “How to adjust the sound quality of earphones”. During the initial encounter, the *Analysis Agent* needs to query the user for the earphones’ model. Once the user provides the information, the context library integrates it, as described in Appendix B.2.

In subsequent instances of similar tasks, the agent will proactively ask the user if he/she needs to “adjust the sound quality of HUAWEI Freebuds Pro”, requiring only a simple confirmation from the user.

1. The first attempt.



2. The second attempt.

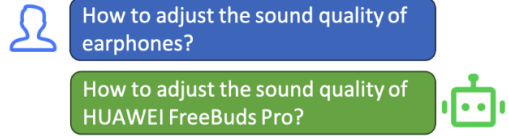


Fig. 25. Case 3: An example of auto-completing the textual prompt.

If the user’s context changes, such as a different earphone model, the context library may provide incorrect information. In such cases, the user can reject the agent’s recommendation, and the agent will engage in a dialogue with the user to correct and then update the context library.

B.3.4 Case 4. Case 4 demonstrates the optimization of the *Parsing Agent* by the instruction set.

Suppose the current step description is “copy the title”, which is a complex instruction that should be decomposed into two instructions:

- <type: longclick, object: title widget> (Figure 26(a))
- <type: click, object: “copy”> (Figure 26(b))



Fig. 26. Case 4: The instruction is a complex copy command that requires first long-pressing the title and then selecting “copy” from the options that pop up.

During the first execution, since it is not included in the basic instruction definition, the *Parsing Agent* can only generate <type: copy, object: “title”>. Consequently, the *Grounding Agent* cannot match the corresponding operation and must rely on the “add” strategy or user demonstration to successfully execute the task. After task completion, the user maps the original compound instruction to the generated decomposed instructions, and the instruction set stores this correspondence for future use.

In subsequent executions, the *Parsing Agent* will automatically decompose the compound instruction by finding similar examples in the instruction set. Then it will allow the *Grounding Agent* to potentially find the target widget directly through heuristic rules. Even if it cannot, it will reduce the difficulty of LLM reasoning in the *Grounding Agent*.

If the app undergoes significant updates, changing the execution steps of this compound instruction, the instructions generated by the *Parsing Agent* may be incorrect. Consequently, the

Grounding Agent will rely on the LLM to redeploy strategies (redirect, add, skip, expand). If it encounters a block, it will require user demonstration.

B.3.5 Case 5. Case 5 demonstrates the optimization of the *Grounding Agent* and the *Assessment Agent* by the mobile interaction graph.

Suppose the current step description is “share the link with a friend”, and the current UI is shown in Figure 27. There are four interactive widgets available for operation from left to right. Suppose the mobile interaction graph already contains the resulting pages after triggering these widgets (with the mobile semantics module in the *Grounding Agent* generating textual descriptions for each page):

- 0 → search result page
- 1 → forwarding list
- 2 → cart popup
- 3 → tools page

As shown in Figure 20 and Figure 21, this information is incorporated into the prompts for the *Grounding Agent* and the *Assessment Agent*, aiding them in making accurate predictions and judgments.

If a significant app update alters the page transition relationships, the mobile interaction graph might provide incorrect information, potentially leading both agents to erroneous decisions. If multiple erroneous attempts occur, the *Assessment Agent* will request user intervention.



Fig. 27. Case 5: The current UI contains 4 candidate widgets. The agents will consider various information for each widget, including the triggered future pages, to make the final decision.

C USER EVALUATION

C.1 Objective Results

The impact of methods and participant groups on task completion time is shown in Table 14.

Table 14. Impact of different methods and participant groups on task completion time (in seconds).

	Manual	Prompt2Task (First Time)	Prompt2Task (Second Time)
Unskilled	110.47 (sd = 54.86)	249.45 (sd = 84.47)	33.11 (sd = 6.14)
Skilled	88.86 (sd = 55.51)	212.09 (sd = 83.66)	32.49 (sd = 7.83)

C.2 Subjective Feedback

After the participants completed the experiment, we conducted interviews with them. The results of the 7-point Likert scale are shown in Table 15.

Table 15. Subjective feedback.

Statements	Results
Providing feedback to the system is effortless.	5.42 (sd=1.16)
You can finish the task easily.	6.33 (sd=0.65)
You find the time taken to be acceptable.	5.83 (sd=1.47)
The system’s decisions are sufficiently intelligent.	6.50 (sd=0.52)
The system’s requests for user assistance are appropriately made.	6.08 (sd=0.51)
You can feel free to query.	6.67 (sd=0.65)
The experience is good.	6.25 (sd=1.06)
You are willing to use our Prompt2Task.	6.58 (sd=0.67)